University of Cape Town

CSC200SS: Computer Science

Author: George Goodey

October 30, 2026

Contents

1 Parallel and Concurrent Programming **4**

1.1 What is parallel computing? . . . 4
1.1.1 Changing a core assumption . . . 4
1.1.2 Parallelism versus concurrency . . . 4
1.1.3 Parallel algorithm example . . . 4
1.1.4 Concurrency example . . . 5
1.1.5 Restaurant analogy . . . 5
1.1.6 Race conditions . . . 6
1.1.7 Non-determinism . . . 6
1.1.8 When is concurrent programming necessary? . . . 7
1.1.9 Moore's law . . . 7
1.1.10 Concurrency does not require multiple cores . . . 7
1.1.11 High performance computing (HPC) . . . 7
1.2 The shared memory programming model . . . 8
1.2.1 The shared memory model . . . 8
1.2.2 Cache . . . 9
1.2.3 Processes . . . 9
1.2.4 Threads . . . 10
1.2.5 Parallel program . . . 10
1.2.6 Sequential program state . . . 11
1.2.7 Programming model: shared memory . . . 11
1.2.8 How threads run . . . 12
1.2.9 Asyncrony . . . 12
1.2.10 Other parallel programming models . . . 12
1.3 Multi-threading in Java . . . 13
1.3.1 Java is always multi-threaded . . . 13
1.3.2 Basic threads in Java . . . 14
1.3.3 Thread class . . . 14
1.3.4 Runnable interface . . . 15
1.3.5 start() vs. run() . . . 15
1.3.6 Thread vs. Runnable example . . . 15
1.3.7 Thread life cycle . . . 16
1.3.8 Stopping a thread . . . 17
1.3.9 Java executors for parallel programming . . . 17
1.3.10 Hello world with Fork/Join . . . 17
1.4 Parallel programming in Java . . . 18
1.4.1 Basic parallel problem . . . 18
1.4.2 Parallelism idea 1: Normal threads (naive) . . . 19
1.4.3 Parallelism idea 2: Divide-and-conquer (best) . . . 20
1.4.4 Fork/Join framework . . . 21
1.5 Parallel algorithms . . . 22
1.5.1 Divide-and-conquer: reductions . . . 22
1.5.2 Divide-and-conquer: maps . . . 22
1.5.3 Parallelizing image processing algorithms . . . 23
1.5.4 Timestep simulations . . . 24
1.5.5 Map vs. stencil . . . 25

1.5.6 Parallel prefix-sum . . . 26
1.5.7 Pack/filter, stream compaction . . . 27
1.5.8 Quicksort . . . 28
1.6 Parallel performance . . . 28
1.6.1 Demonstrating parallel performance . . . 28
1.6.2 Speedup . . . 28
1.6.3 Parallel efficiency . . . 29
1.6.4 Theoretical analysis of expected speedup . . . 29
1.6.5 Limits on T_P . . . 30
1.6.6 The DAG (cost graph) . . . 30
1.6.7 Divide-and-conquer parallel algorithms . . . 31
1.6.8 Why measured speedup falls below the ideal . . . 31
1.6.9 Where does scaling stop? . . . 32
1.6.10 Amdahl's law . . . 32
1.6.11 Gustafson's law . . . 33
1.6.12 Scaling with problem size . . . 33
1.6.13 Grid simulations . . . 34
1.6.14 Two kinds of scaling . . . 34
1.7 Concurrency and mutual exclusion . . . 35
1.7.1 Parallel vs. concurrent programming . . . 35
1.7.2 Synchronization and mutual exclusion . . . 36
1.7.3 Race conditions and atomic classes . . . 36
1.7.4 Compound actions and bad interleavings . . . 38
1.7.5 Locks, monitors, and reentrancy . . . 40
1.7.6 Spin-waiting . . . 41
1.8 Concurrency and thread safety . . . 42
1.8.1 Thread safety fundamentals . . . 42
1.8.2 Monitors and compound actions . . . 43
1.8.3 Visibility and volatile variables . . . 44
1.8.4 Java synchronizers . . . 44
1.8.5 Reordering and memory model . . . 45
1.8.6 The grand compromise . . . 46
1.9 Condition variables . . . 46
1.9.1 Busy waiting . . . 46
1.9.2 Java monitors, sets, thread states . . . 47
1.9.3 Condition variable methods and pattern implementations . . . 48
1.9.4 Correctness rules and liveness hazards . . . 48
1.10 Thread liveness . . . 49
1.10.1 Concurrent correctness properties . . . 49
1.10.2 Deadlocks and deadly embrace . . . 49
1.10.3 Deadlock prevention strategies . . . 49

2 Computer Architecture **50**
2.1 Introduction . . . 50
2.1.1 What is architecture? . . . 50
2.1.2 Assembly programming . . . 50
2.1.3 Components of a computer . . . 50
2.1.4 Measuring performance . . . 50

2.1.5 Performance fallacies . . . 51
2.2 MIPS . . . 51
2.2.1 Von Neumann architecture . . . 51
2.2.2 Hierarchy of programming languages . . . 52
2.2.3 Assembly . . . 53
2.2.4 Instruction philosophies . . . 53
2.2.5 MIPS . . . 54
2.2.6 Assembler directives . . . 55
2.2.7 Labels . . . 55
2.2.8 Registers . . . 55
2.2.9 Memory . . . 56
2.2.10 Stack . . . 58
2.2.11 Data transfer instructions . . . 59

3 Theory of Computation **59**
3.1 Regular Languages . . . 59
3.1.1 Finite automata . . . 59
3.1.2 Strings and languages . . . 60
3.1.3 Regular expressions . . . 61
3.1.4 Nondeterministic finite automata . . . 61
3.1.5 Converting NFAs to DFAs . . . 63
3.1.6 Closure properties for regular languages . . . 63
3.1.7 Converting a regular expression to NFA . . . 64
3.1.8 Generalized NFA . . . 66
3.1.9 Converting a GNFA to a regular expression . . . 67
3.1.10 Non-regular languages . . . 67
3.1.11 Proving non-regularity . . . 68
3.2 Context-free languages . . . 70
3.2.1 Context-free grammars . . . 70
3.2.2 Ambiguity . . . 70
3.2.3 Pushdown automaton . . . 71
3.2.4 Converting CFGs to PDAs . . . 72
3.2.5 Equivalence of CFGs and PDAs . . . 72
3.2.6 Chomsky normal form (CNF) . . . 73
3.2.7 Proving languages are not context free . . . 74
3.3 Turing Machines . . . 76
3.3.1 Turing machines . . . 76
3.3.2 Determinism . . . 76
3.3.3 Instantaneous descriptions . . . 77
3.3.4 Language accepted by a turing machine . . . 77

1 Parallel and Concurrent Programming

1.1 What is parallel computing?

1.1.1 Changing a core assumption

So far, all algorithms have assumed that one thing happens at a time. This is traditional sequential programming. We have to change the core assumption. Multiple activities may be in process concurrently, and some may execute simultaneously in parallel. Parallel and concurrent programming remove the requirement that all computational steps belong to one global serial order.

Definition: Commonly used terms

In most conventional usage,

- **Parallel** programming focuses on arranging that multiple activities co-occur.
- **Distributed** programming focuses on arranging thta activities occur in different places.
- **Concurrent** programming focuses on interactions of ongoing activities with each other.

However, all three terms may apply in most contexts.

1.1.2 Parallelism versus concurrency

Parallel program:
Use extra computational resources to solve a problem faster by performing multiple computations simultaneously.

Concurrent program:
Allow multiple activities to be in process during the same period of time.

1.1.3 Parallel algorithm example

Here is the pseudo-code for an array sum algorithm:

```
public int parallelSum(array, workers):
    partial = new array[workers];

    PARALLEL FOR worker = 0 to workers - 1 {
        lo = worker * array.length / workers;
        hi = (worker + 1) * array.length / workers;
        partial[worker] = sequentialSum(array, lo, hi);
    }

    return sequentialSum(partial, 0, workers);
```

Parallel workers

With p workers, the theoretical upper bound for the parallel summation phase is approximately p, but the actual speedup will usually be slower because of:

1. thread-management overhead,
2. memory bandwidth,
3. load imbalance, and
4. the final sequential combination.

1.1.4 Concurrency example

For example, multiple operations on a hash table. What could go wrong if two threads update the same bucket at the same time?

1. Lost updates;
2. Corruption of a bucket chain or tree;
3. Duplicate entries despite a uniqueness invariant;
4. A lookup observing a partially constructed state;
5. Resizing while another operation uses the old table;
6. etc.

Here is the pseudo-code for a shared hash table:

```
public void insert(key, value):
    bucket = findBucket(key)

    /* Acquire protection for bucket */
    /* Perform insertion */
    /* Release protection for bucket */
```

Shared hash table

- Prevent bad interleaving (correctness)
- But allow some concurrent access (performance)

1.1.5 Restaurant analogy

A serial program is a recipe for one cook. The cook does one thing at a time (sequential), maybe switching between tasks (time-slicing). Options for lots of cooks:

- use helpers to assist with a big task, like slicing lots of potatoes for hot chips **(data parallelism)**.
- dedicate helpers to different taks, such as beating eggs, slicing onions, washing up **(concurrency)**.

- have a list of taks and assign them to workers as they are free **(worker pool)**.
- But too many chefs mean too much time coordination which leads to them bumping into each other etc.

Often there are multiple steps that can be done at the same time with access to more cooks. More cooks require extra coordination. A cook may have to wait for another cook to finish something. There are limited resources.

- If you only have one oven, two cooks won't be able to bake cakes at different temperatures at the same time.
- Some recipes will require exclusive use of the oven.

One approach is to allow access to the oven, but only one at a time (cakes can flop). Cooks must wait until the oven is free.

```
if (!ovenBusy) {
    ovenBusy = true;
    bakeCake();
    ovenBusy = false;
}
```

Pseudo-code issue

This specific pseudo-code is fine for a single-threaded program, but will it work for a multi-threaded program?

1.1.6 Race conditions

Definition: Race condition

A **race condition** is an error where the correctness of a program depends on the relative timing or interleaving of concurrent operations.

The idea is that the events race each other. A particular execution may appear correct even though the race condition is present. Uninteded race conditions are incredibly common in concurrent programs.

1.1.7 Non-determinism

Deterministic:
In a simple sequential program, instructions are executed in a fixed order determined by the program and its input. The execution of one procedure does not overlap in time with another.

Non-deterministic:
In concurrent programs, there are many possible interleavings. The exact interleaving is generally not predictable. Non-deterministic scheduling does not imply an incorrect or unpredictable final result, but a correct concurrent program should satisfy its specification under every permitted interleaving.

1.1.8 When is concurrent programming necessary?

Complex systems like:

- **operating systems**,
- **web browsers**,
- **real-time** systems,
- **event-based** implementations of graphical user interfaces,
- multiuser **games**, **chat** and **e-commerce**

handle mutiple things happening at once and are therefore multi-threaded. These systems must respond to multiple ongoing activities. Their tasks may be interleaved on one core or executed in parallel on several cores.

1.1.9 Moore's law

Between 1980-2005, Moore's law held:

- clock speeds of desktop computers doubled every 18-24 months
- exponentially faster at running sequential programs

But, it eventually reached

- **Power wall:** increasing frequency and transistor activity produce excessive power use and heat.
- **Memory wall:** processor performance improved faster than memroy-access latency and bandwidth.

1.1.10 Concurrency does not require multiple cores

On one logical processor, runnable threads take turns through scheduling and pre-emption. Their executions are interleaved. On multiple logical processors, several threads may execute simultaneously. The operating system and runtime control processor scheduling.

1.1.11 High performance computing (HPC)

HPC uses cutting-edge parallel computers to solve "grand-challenge" (computational) problems. Programmers write programs to solve such computationally large problems that it would take years, or centuries, for one computer to finish it.

Deep learning:
The modern growth of deep learning was enabled by:

- large training datasets;
- algorithmic advances;
- GPUs and specialized accelerators;

- efficient parallel software frameworks;
- large-scale distributed computing.

Weather forecasting and climate modelling:
Weather models divide the atmosphere and oceans into a three-dimensional grid. Each grid cell requires repeated calculations of temperature, pressure, moisture, wind, radiation, and fluid movement. Cells interact with neighboring cells, so the calculations is both highly parallel and communication-intensive.

Molecular modelling:
Molecular modelling simulates the behavior and interactions of molecules to try to understand their properties. It is widely used in drug development.

Animations and visual effects:
Typically, animation studios need HPC systems. Computer graphics must be rendered into high-resolution images. Rendering turns a virtual 3D scene into a 2D image, and requires a lot of computational power.

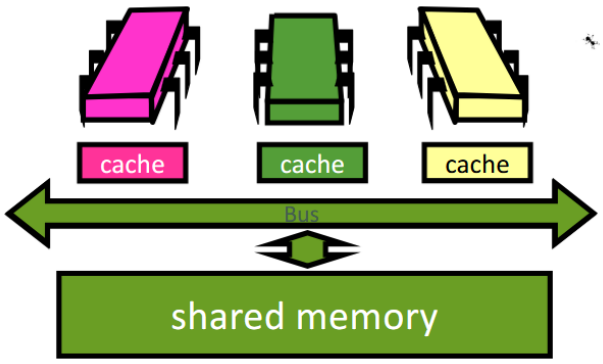
Note

While HPC as a concept should be known, the "in-between" bits for deep learning, forecasting, molecular modelling, etc. are not necessarily examinable.

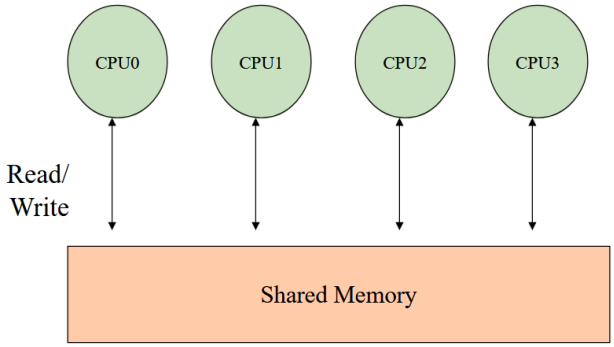
1.2 The shared memory programming model

1.2.1 The shared memory model

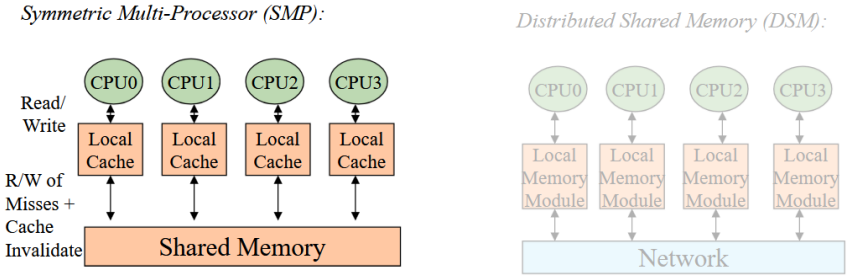
All memory is placed into a single (physical) address space. Processors are connected by some form of interconnection network. There is a single virtual address space across all of memory. Each processor can access all locations in memory.



The ideal picture of shared memory:



versus, the actual architecture of shared memory systems:



1.2.2 Cache

A memory cache, also called a "CPU cache," is a memory bank that stores recently used instructions and data close to the processor.

- Is had faster static RAM (SRAM) chips than the dynamic RAM (DRAM) used for main memory.
- The cache allows instructions to be executed and data to be read and written at higher speed.

Modern chips can have multiple caches (L1, L2, L3):

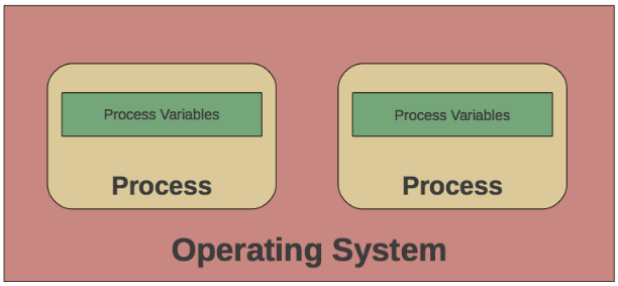
- The L1 cache is the fastest.
- Each subsequent cache is slower but larger than L1.
- Instructions and data are staged from main memory to L3 to L2 to L1 to the processor.
- One multicore chips, the L3 cache is generally shared among all the processing chips.

Why do caches matter for PCP? If two cores repeatedly modify the same variable, their caches continually invalidate one another.

1.2.3 Processes

Definition: Process

A **process** is an operating system unit of resource allocation both for CPU time and for memory.



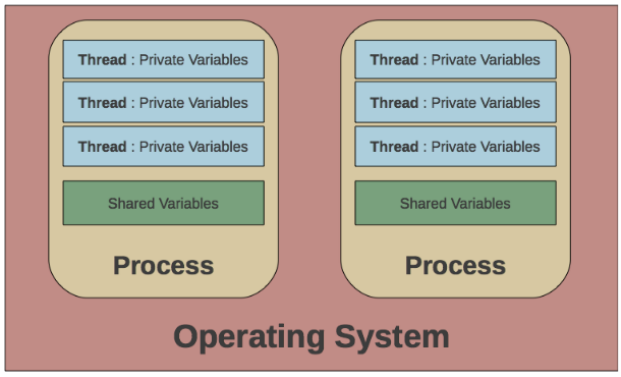
A process is represented by its code, data, and the state of the machine registers. The data/memory typically contains:

- code
- global/static data
- heap
- stack

Generally, each process in an operating system has its own address space.

1.2.4 Threads

Process given internal concurrency with multiple (lightweight) threads. This has multiple threads of control. A process with multiple (lightweight) threads of control has multiple stacks, one for each thread, but access to shared memory too.



1.2.5 Parallel program

The shared memory model has multiple explicit threads running concurrently. Threads can:

- perform multiple computations in parallel;
- perform separate simultaneous activities;
- communicate implicitly by reading and writing shared memory.

Warning

Can be dangerous if you don't protect your variables correctly!

1.2.6 Sequential program state

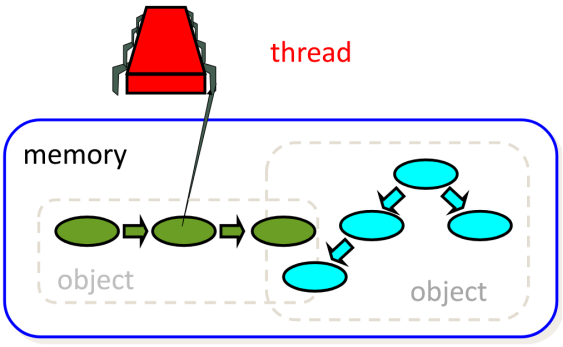
A running serial program has:

- **One program counter**
- **One call stack**
  - Each stack frame holds the local variables for a method call that has started but not yet finished.
  - Calling a method pushes a new frame and returning from a method pops a frame.
  - Call stacks are why recusion is not "magic."
- **Objects**
  - Objects are created by calling **new**.
  - We call the memory that holds all the objects the heap.
- **Static fields of classes.**

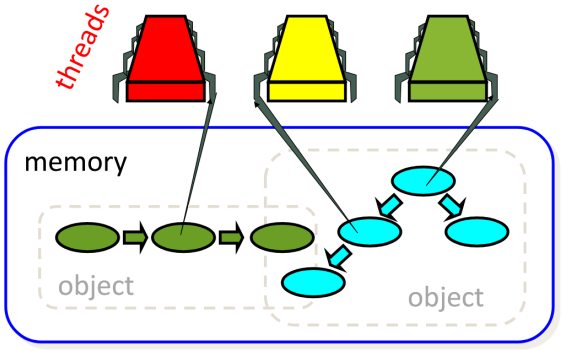
1.2.7 Programming model: shared memory

Each thread has its own program counter, call stack, and local variables. All threads share one collection of objects and static fields. Static fields of classes are also shared by all threads. Threads communicate through shared objects (implicit communication). Threads communicate by writing data that another thread later reads.

Sequential computation:



Concurrent computation:



1.2.8 How threads run

As the programmer, you create the threads. The operating system scheduler determines how and when those threads are run on the available processors. Unless you are writing the scheduler, **you do not have control over this**. You don't how how many processors/cores your program will use (you can guess). You don't how the order in which threads will execute (you can not guess).

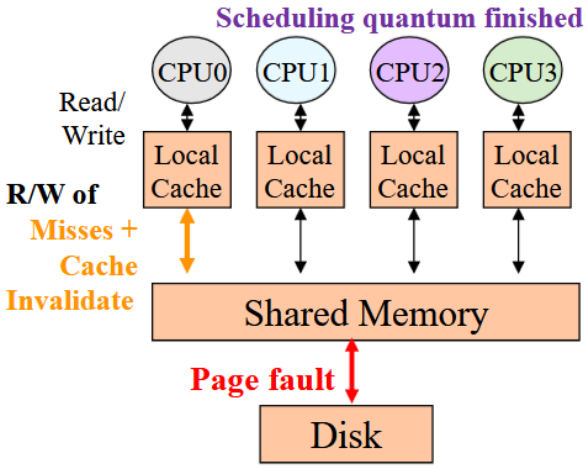
Super important

Never write code that depends on thread execution order!

1.2.9 Asyncrony

Threads are subject to sudden unpredictable delays:

- **Cache misses** (short: nanoseconds)
- **Page faults** (long: milliseconds)
- **Scheduling quantum finished** (really long: milliseconds)
- **Garbage collection** (really, really long: milliseconds to seconds)



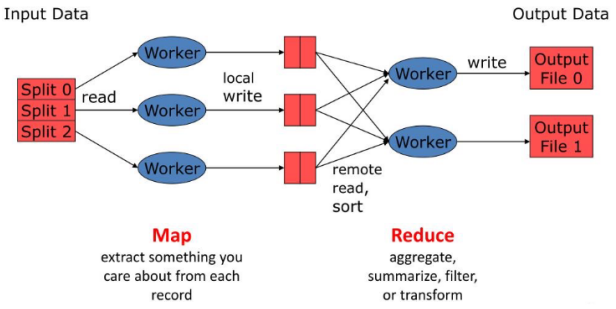
1.2.10 Other parallel programming models

Message passing:

- Explicit threads/processes, each with their own objects/data.
- Communication is via explicitly sending/receiving messages, containing copies of the data (each process has its own memory).
- Most common model on HPC systems.

MapReduce:

- Data parallelism concept from functional programming languages like LISP.
- Have primitives for things like "apply function to every element of an array in parallel."
- MapReduce was developed by Google and its programming model has since been adopted by many software frameworks, e.g. Apache's open-source Hadoop.



Golang:
Go provides lightweight threads (goroutines) and channels for communication.

Note

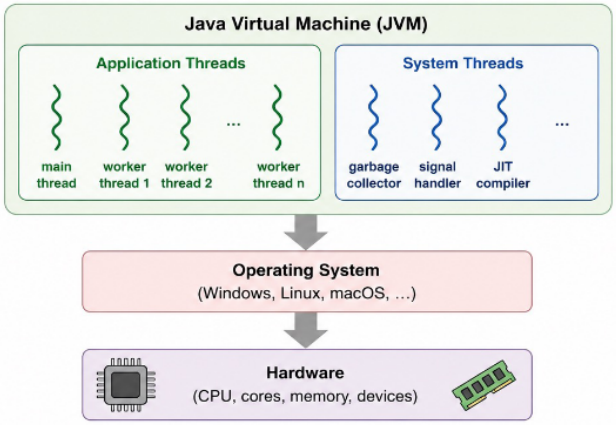
Don't worry too much about the finer details when going through this section. The most important takeaway here is that there are alternatives to the shared memory model.

1.3 Multi-threading in Java

1.3.1 Java is always multi-threaded

The Java Virtual Machine (JVM) executes as a process under the operating system. In Java, every program has more than one thread. It starts with one thread, called the main thread. The main thread can then create additional threads. The JVM also creates system threads to perform tasks such as:

- garbage collection,
- signal handling,
- JIT compilation,
- etc.



1.3.2 Basic threads in Java

All threads are associated with an instance of the class **java.lang.Thread**. There are two basic strategies for using the Thread class:

1. create a Thread object each time the application needs to initiate an asynchronous task, or
2. pass the application's tasks to an executor (**java.util.concurrent**) that will launch and manage the threads. This is the standard approach for parallelization. Uses a high-level API to manage a thread pool for large-scale applications that run on multi-processor and multi-core systems.

There are two ways to create a basic thread in Java:

1. Extend the Thread class (**java.lang.Thread**). Easy for simple programs.
2. Implement the Runnable interface in an object and pass that to a thread's constructor (**java.util.Runnable**). This is the more general and flexible approach.

1.3.3 Thread class

A Thread class manages a single sequential thread of control. Threads may be created and deleted dynamically. The Thread class executes instructions from its method **run()**. The actual code executed depends on the implementation provided for **run()** in a derived class.

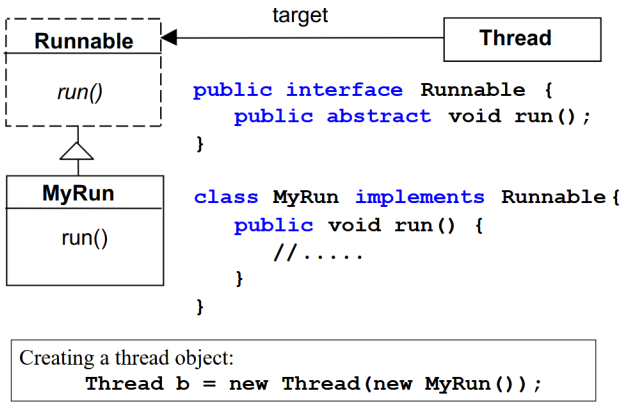
```
class MyThread extends Thread {
      public void run() {
            //......
      }
}
```

Creating a thread object:

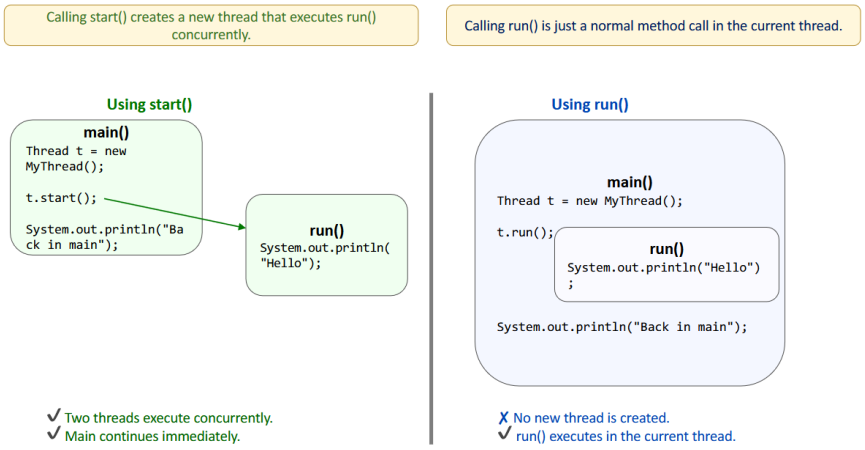
```
Thread a = new MyThread();
```

1.3.4 Runnable interface

Since Java does not permit multiple inheritance, it is sometimes more convenient to implement the **run()** method in a class not derived from Thread, but from the interface Runnable. The Runnable interface defines a single method, **run()**, containing the code executed in the thread.



1.3.5 start() vs. run()



Note

TL;DR Always use **start()**.

1.3.6 Thread vs. Runnable example

Here is an example of a program using Thread:

```
public class HelloThread extends java.lang.Thread {
    private int i;
    HelloThread(int i) { this.i = i; }

    public void run() {
        System.out.println("Thread " + i + " says hi");
        System.out.println("Thread " + i + " says bye");
    }

    public static void main(String[] args) {
        for(int i=1; i <= 10; ++i) {
            HelloThread c = new HelloThread(i);
            c.start();
        }
    }

}
```

Constructor

This is what each thread does.

This is what makes the thread run

When this program runs, it will print 10 lines of output, one of which is:

```
Thread 10 says hi
```

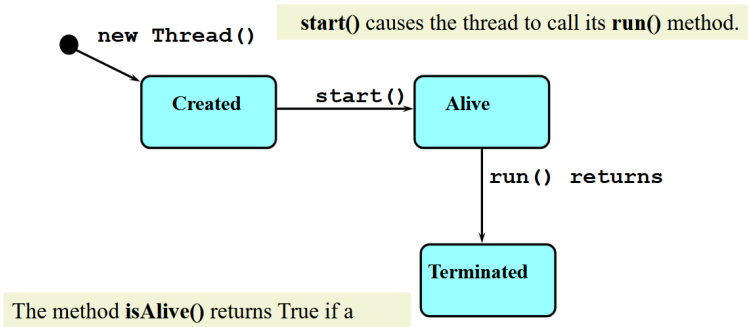
versus, the same program instead using Runnable:

```
public class HelloInterface implements Runnable {
    private int i;
    HelloInterface(int i) { this.i = i; }

    public void run() {
        System.out.println("Thread " + i + " says hi");
        System.out.println("Thread " + i + " says bye");
    }

    public static void main(String[] args) {
        for(int i=1; i <= 10; ++i) {
            Thread c = new Thread(new HelloInterface(i));
            c.start();
        }
    }
}
```

1.3.7 Thread life cycle



start() causes the thread to call its **run()** method.

The method **isAlive()** returns True if a thread has been started but not terminated.

Once terminated, **it cannot be restarted.**

1.3.8 Stopping a thread

Most of the time, we allow threads to stop by running to completion. Sometimes we want to stop threads sooner, e.g. when the user cancels an operation. It is not very easy to get threads to stop safely, quickly, and reliably. **Thread.stop()** and **Thread.suspend()** are too dangerous and was deprecated. Instead, ask the thread to stop by using an interupt.

1.3.9 Java executors for parallel programming

Executor objects in Java distribute tasks to worker threads in a thread pool. Thread pools reduce thread creation overhead since allocating and deallocating many thread objects require significant memory management. There are three executor interfaces in Java. We primarily focus on **Fork/Join**. The Fork/Join executor is designed for divide-and-conquer algorithms especially thos that are recursive. It uses the "work-stealing" algorithm, where idle threads can steal tasks from busy threads. **ForkJoinPool** is the main thread pool class. There are two types of tasks sent to the pool.

- **RecursiveAction** has no return value, and
- **RecursiveTask** returns a value.

1.3.10 Hello world with Fork/Join

Here are two versions of the same program. The first has race conditions and the second does not. The first version:

Hello World, with *Fork-Join* (and a race condition...)

```
import java.util.concurrent.ForkJoinPool;
import java.util.concurrent.RecursiveAction;

// RecursiveAction has no return value, RecursiveTask returns a value
public class HelloMany extends RecursiveAction  {
    int greetings; // arguments
    int offset;

    HelloMany(int g, int start) {
      greetings=g;
      offset=start;
    }

    protected void compute(){
       if((greetings) <=1) { //only one task left, do it. This cutoff would be bigger for proper programs
           System.out.println("hello"+offset );
       }
         else {
               int split=(int) (greetings/2.0);
               //split work into two
               HelloMany left = new HelloMany(split,offset);  //first half
               HelloMany right= new HelloMany(greetings-split,offset+split ); //second half
               left.fork(); //give first half to new thread
               right.compute(); //do second half in this thread
            }
     }
    public static void main(String[] args) {
        HelloMany sayhello = new HelloMany(50,0); //the task to be done, divide and conquer
        ForkJoinPool pool  = new ForkJoinPool(); //the pool of worker threads
        pool.invoke(sayhello); //start everything running - give the task to the pool
     }
    }
```

versus, the second version:

```
import java.util.concurrent.ForkJoinPool;
import java.util.concurrent.RecursiveAction;

// RecursiveAction has no return value, RecursiveTask returns a value
public class HelloMany extends RecursiveAction  {
    int greetings; // arguments
    int offset;

    HelloMany(int g, int start) {
      greetings=g;
      offset=start;
    }
    protected void compute(){
       if((greetings) <=1) { //only one task left, do it. This cutoff would be bigger for proper programs
          System.out.println("hello"+offset );
       }
         else {
              int split=(int) (greetings/2.0);
              //split work into two
              HelloMany left = new HelloMany(split,offset);  //first half
              HelloMany right= new HelloMany(greetings-split,offset+split ); //second half
              left.fork(); //give first half to new threas
              //left.join(); // what will this do if included?
              right.compute(); //do second half in this thread
              left.join(); //corrected!!
         }}
    public static void main(String[] args) {
        HelloMany sayhello = new HelloMany(50,0); //the task to be done, divide and conquer
        ForkJoinPool pool  = new ForkJoinPool(); //the pool of worker threads
        pool.invoke(sayhello); //start everything running - give the task to the pool
     }
    }
```

Hello World, with *Fork-Join* (race condition removed)

1.4 Parallel programming in Java

1.4.1 Basic parallel problem

Let's take the example of summing the elements of an array. An $O(n)$ sequential solution to this problem is trivial:

```
public long sum(long[] arr) {
    long ans = 0;
    for (int i=0; i<arr.length; i++) {
        ans += arr[i];
    }
    return ans;
}
```

Note

This problem is to illustrate the concept of parallelization. It's not an ideal problem to parallelize...

Benchmarking the results with an array filled with 1's, we get:

```
Adding an array of 10000000 integers serially took 4.151084 ms
Adding an array of 10000000 integers serially took 3.539417 ms
Adding an array of 10000000 integers serially took 2.566125 ms
Adding an array of 10000000 integers serially took 2.603666 ms
Or...
Adding an array of 100000000 integers serially took 27.093584 ms
Adding an array of 100000000 integers serially took 25.09275 ms
Adding an array of 100000000 integers serially took 23.096376 ms
Adding an array of 100000000 integers serially took 23.286165 ms
```

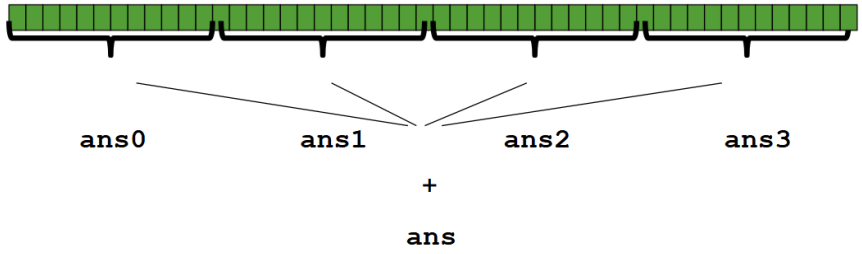
Multiple repetitions are needed because of:

- JVM startup and class loading
- JIT compilation
- cache state
- operating-system scheduling
- garbage collection

Parallel programming is only really worth the effort for programs that take too long to run serially...this is too small

1.4.2 Parallelism idea 1: Normal threads (naive)

Let's suppose 4 cores can execute work simultaneously. The idea is: have 4 threads simultaneously sum 1/4 of the array each.



Let's implement the task each thread will execute:

```
public class SumThread extends Thread {
        private int lo; // arguments
        private int hi;
        private long[] arr;
        int ans = 0; // result

        public SumThread(long[] arr2, int l, int h) {
                lo=l; hi=h; arr=arr2;
        }

        public void run() {
          for(int i=lo; i < hi; i++)
            ans += arr[i];
        }
}
```

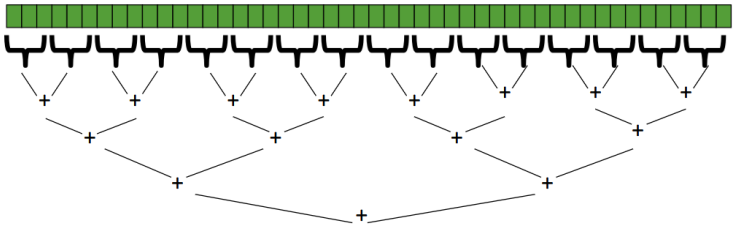
Because we must override a no-arguments/no-result **run()** method, we use fields/variables to communicate across threads. After implementing the main thread section correctly, it should look something like this:

```
static long sum(long[] arr, int numTs) throws  InterruptedException{
                long ans = 0;
                SumThread[] ts = new SumThread[numTs];
                for(int i=0; i < numTs; i++){  // do parallel computations
                        ts[i] = new SumThread(arr,(i*arr.length)/numTs,
                                ((i+1)*arr.length)/numTs);
                        ts[i].start();
                }

                for(int i=0; i < numTs; i++) {
                        ts[i].join();
                  ans += ts[i].ans;
                }
                return ans;
              }
```

1.4.3 Parallelism idea 2: Divide-and-conquer (best)

This problem is fairly straightforward to implment using a divide-and-conquer algorithm. We use the parallelism for the recursive calls. The result-combining is done in parallel as well, which is mor efficient.



Using divide-and-conquer with platform threads:

```
public class SumThread extends Thread {
    int lo; // arguments
    int hi;
    int[] arr;
    static  int SEQUENTIAL_CUTOFF=500;

    int ans = 0; // result

    SumThread(int[] a, int l, int h) {
      lo=l; hi=h; arr=a;
    }
```

```
public void run() {
 if((hi-lo) < SEQUENTIAL_CUTOFF)
   for(int i=lo; i < hi; i++)
                ans += arr[i];
 else {
  try {
   SumThread left = new SumThread(arr,lo,(hi+lo)/2);
   SumThread right= new SumThread(arr,(hi+lo)/2,hi);
                // order of next 4 lines
                // essential - why?
    left.start();
    right.run();  //Q: why run and not start?
    left.join();
     ans = left.ans + right.ans;
  }
 catch (InterruptedException e) {
          e.printStackTrace();
}
}
}}
```

1.4.4 Fork/Join framework

Important distinction

A **task** describes **work**. A **worker** thread executes **work**.

Note

ForkJoinTasks (either RecursiveAction or RecursiveTask) are given to a **ForkJoinPool** (a pool of threads).

There are two ways to creatre a ForkJoinPool.

Option 1:

```
static final ForkJoinPool fjPool = new ForkJoinPool();
```

Here, **ForkJoinPool()** creates a pool with parallelism equal to **Runtime.availableProcessors()**, i.e. the total number of processors in your machine. Importantly, you can specify the parallelism with

```
new ForkJoinPool(int p)
```

Option 2:

```
static final ForkJoinPool fjPool = new ForkJoinPool.commonPool();
```

Here, **commonPool()** creates a pool with parallelism equal to **Runtime.availableProcessors()-1**, i.e. one less than the total number of processors in your machine. Using this, we can implement the final version of our program:

```
public class SumArray extends RecursiveTask<Long> {
  private final int lo;
  private final int hi;
  private final long[] arr;

  static int SEQUENTIAL_CUTOFF = 5000;

  SumArray(long[] arr, int lo, int hi) {
    this.lo = lo;
    this.hi = hi;
    this.arr = arr;
  }

  @Override
  protected Long compute() {
    if (hi - lo < SEQUENTIAL_CUTOFF) {
      long ans = 0;
      for (int i = lo; i < hi; i++) {
        ans += arr[i];
      }
      return ans;
    }

    int mid = (lo + hi) / 2;
    SumArray left = new SumArray(arr, lo, mid);
    SumArray right = new SumArray(arr, mid, hi);

    left.fork();
    long rightAns = right.compute();
    long leftAns = left.join();

    return leftAns + rightAns;
  }
}
```

When is Fork/Join really useful?

- When you are doing the parallel computation many times.
- When threads have a lot to do.
- When tasks or subproblems require different amounts of work, we may encounter load imbalance. In this case, we want idle workers to steal queued tasks from busy workers.

1.5 Parallel algorithms

1.5.1 Divide-and-conquer: reductions

Reduction operations produce a signle answer from a collection via an associative operator, e.g. **max**, **count**, **left-most**, **sum**, etc. Recursive results don't have to be single numbers of strings. They can be arrays or objects with multiple fields.

1.5.2 Divide-and-conquer: maps

A map independently transforms every input element, producing one output element for each input element. There is no combining of results. A canonical example: vector addition

```
public int[] vector_add(int[] arr1, int[] arr2) {
    assert (arr1.length == arr2.length);
    result = new int[arr1.length];

    for (int i=0; i<arr1.length; i++) {
        result[i] = arr1[i] + arr2[i];
    }

    return result;
}
```

Maps with the Fork/Join framework:

```
public class VecAdd extends RecursiveAction {
  int lo; int hi; int[] res; int[] arr1; int[] arr2;
  static final int SEQUENTIAL_CUTOFF=1000;

  VecAdd(int lo, int hi, int[] res, int[] arr1, int[] arr2) {
    this.lo = lo;
    this.hi = hi;
    this.res = res;
    this.arr1 = arr1;
    this.arr2 = arr2;
  }

  protected void compute(){
    if((hi - lo) < SEQUENTIAL_CUTOFF) {
                    for(int i=lo; i < hi; i++)
      res[i] = arr1[i] + arr2[i];
    } else {
      int mid = (hi+lo)/2;
      VecAdd left = new VecAdd(lo,mid,res,arr1,arr2);
      VecAdd right= new VecAdd(mid,hi,res,arr1,arr2);
      left.fork();
      right.compute();
      left.join();
    }
  }
```

```
  public static void main(String[] args) {
      int size = 10000;

      int[] arr1 = new int[size];
      int[] arr2 = new int[size];
      int[] result = new int[size];

      // Initialise the input arrays.
      for (int i = 0; i < size; i++) {
        arr1[i] = i;
        arr2[i] = 2 * i;
      }
      ForkJoinPool pool = new ForkJoinPool();
      VecAdd task = new VecAdd(0, size, result, arr1,
arr2);
      pool.invoke(task);//execute and join
      // Print a small sample rather than all 10,000
values.
      System.out.println("First ten values:");
      System.out.println(
Arrays.toString(Arrays.copyOf(result, 10)) );
    }
}
```

1.5.3 Parallelizing image processing algorithms

Low-level image processing uses the individual pixel values to modify the image in some way. Image processing operations can be divided into:

- **point operations:** output produced based on the value of a single pixel.
  - thresholding;
  - brightness adjustment;
  - contrast adjustment;
  - color inversion;
  - converting RGB to greyscale;
  - coloring pixels (Mandlebrot set)
- **local operations:** produce output based on a group of neighboring pixels, e.g. smoothing.

public class ParallelThresholding **extends** RecursiveAction { ...

```
protected void work() {
      int R, G, B, brightness, clr;
      int threshold = 128;

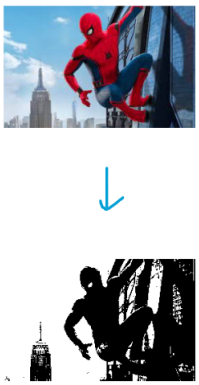
      for (int y = y_start; y < y_stop; y++) {
         for (int x = x_start; x < x_stop; x++) {
            clr = img.getRGB(x, y);

            R = (clr >> 16) & 0xff;
            G = (clr >> 8) & 0xff;
            B = clr & 0xff;

            brightness = (R + G + B) / 3;

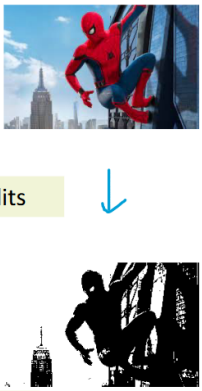
            if (brightness < threshold)
               clr = 0xff000000;  // black
            else
               clr = 0xffffffff;  // white

            dstImg.setRGB(x, y, clr);
         }
      }
   }
```



public class ParallelThreshholding **extends** RecursiveAction { ...

```
protected void compute() {
    if ((x_stop-x_start)*(y_stop-y_start) < sThreshold) {
       work();
       return;
    }
    int width = x_stop - x_start;
    int height = y_stop - y_start;
    ParallelThreshholding left, right;
    if (width >= height) {
       int mid = x_start + width / 2;
       // split left/right
       left = new ParallelThreshholding(img, x_start,mid,
                                y_start, y_stop,  dstImg);
      right = new ParallelThreshholding(img, mid, x_stop,
                                y_start, y_stop, dstImg);
    } else {
       int mid = y_start + height / 2;
       // split top/bottom
       left = new ParallelThreshholding(img, x_start,x_stop,
                                y_start, mid,  dstImg);
        right = new ParallelThreshholding(img, x_start, x_stop,
                                mid, y_stop, dstImg);
    }
    left.fork();
    right.compute();
    left.join();
}
```



Note the differing splits

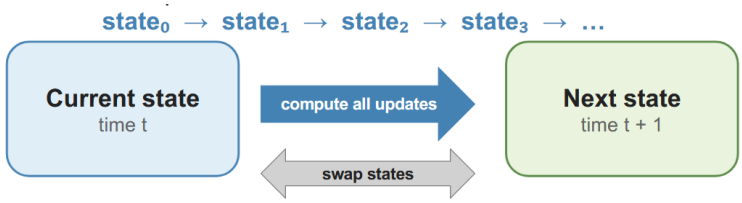
Tasks may safely write into the same destination array only because their output regions do not overlap.

More complex image operation: median smoothing
A mean filter sets each pixel in the image to the average of the currounding pixels, whereas a median filter sets each pixel to the median of the surrounding pixels. A median filter is much better at perserving edges after smoothing.



1.5.4 Timestep simulations

Many simulations model a system that changes through a sequence of discrete timesteps:



Stencil computations are a common way to calculate the next state in a grid-based timestep simulation.

- Each timestep may contain a large amount of parallel work.
- All calculations in timestep t should use the same old state.
- The next timestep cannot begin until timestep t is complete.
- Stop after a fixed number of timesteps or when a termination/convergence condition is reached.

Important

Do not update the current state in place if later calculations in the same timestep depend on it.

Examples of timestep simulations:

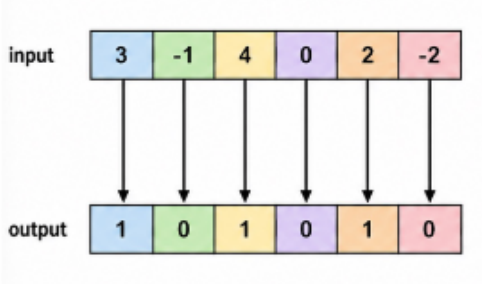
- heat diffusion;
- weather simulations;
- fluid flow;
- molecular simulations;
- astronomical simulations;

- cellular automata;
- wildfire spread.

1.5.5 Map vs. stencil

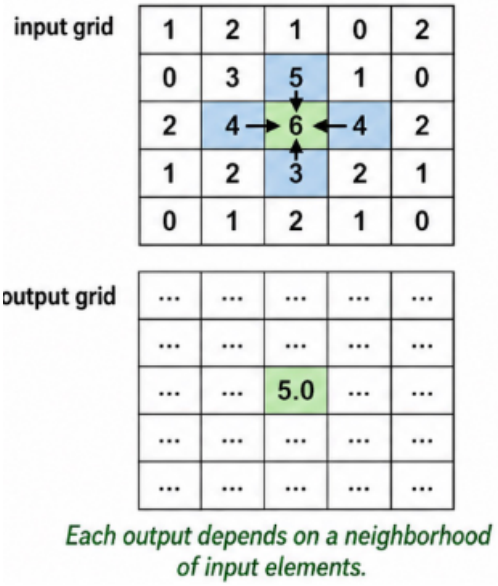
Map:

- Output depends on one input element.
- Example: image thresholding.
- No neighborhood or boundary issues.
- Usually a single pass.
- Directly independent.



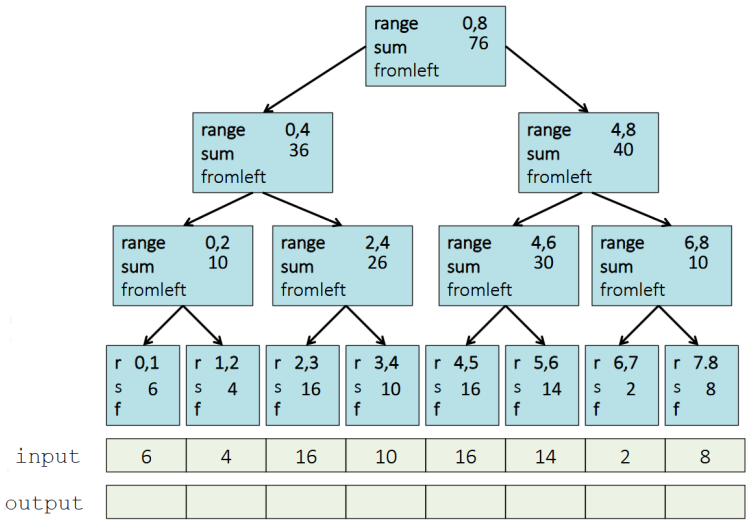
Stencil:

- Output depends on neighboring elements.
- Example: heat diffusion.
- Boundary treatment is required.
- Often repeated over many timesteps.
- Independent only when reading an unchanged state.



1.5.6 Parallel prefix-sum

The parallel prefix-sum does two passes. The first pass builds a tree bottom-up: a reduction pass to compute subtree sums. The second pass traverses the tree top-down: a distribution pass to propogate the accumulated value from the left.

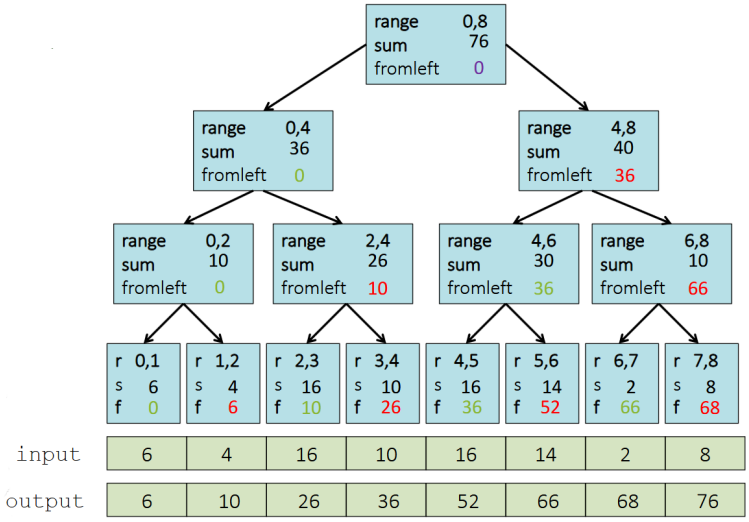


Algorithm: part 1
Build a binary tree where:

- The root has sum of the range $[x, y)$.
- If the range contains more than one element, i.e. $\text{hi} - \text{lo} > 1$:
  - Left child has sum of $[\text{lo}, \text{mid})$.
  - Right child has sum of $[\text{mid}, \text{hi})$.
  - A leaf has sum of $[i, i+1)$, i.e. **input[i]**.

Algorithm: part 2
Pass down a value **fromLeft**:

- Root given a **fromLeft** of 0.
- Node takes its **fromLeft** value and
  - Passes its left child the same **fromLeft**.
  - Passes its right child its **fromLeft** plus its left child's **sum**.
- At the leaf for array position i, **output[i] = fromLeft[i] + input[i]**.



1.5.7 Pack/filter, stream compaction

Given an array **input**, produce an array **output** containing only elements such that $f(x)$ is true. For example:

```
input [17, 4, 6, 8, 11, 5, 13, 19, 0, 24]
 f: x > 10
 output [17, 11, 13, 19, 24]
```

Finding the elements for **output** is easy, but getting them in the right place seems hard. We can use parallel prefix-sum to help us:

1. Parallel map to compute a bit-vector for true vectors.

```
input  [17, 4, 6, 8, 11, 5, 13, 19, 0, 24]
bits   [1,  0, 0, 0,  1, 0,  1,  1, 0,  1]
```

2. Parallel prefix-sum on the bit-vector.

```
bitsum [1,  1, 1, 1,  2, 2,  3,  4, 4,  5]
```

3. Parallel map to produce the output.

```
output [17, 11, 13, 19, 24]
```

Here is the pseudo-code:

```
output = new array of size bitsum[n-1]
for (int i=0; i<input.length; i++) {
    if (bits[i]==1) {
        output[bitsum[i]-1]=input[i];
    }
}
```

1.5.8 Quicksort

Quicksort is very popular sequential sorting algorithm that performs well with:

- Average time: $O(N \log N)$.
- Worst-case time: $O(N^2)$

1.6 Parallel performance

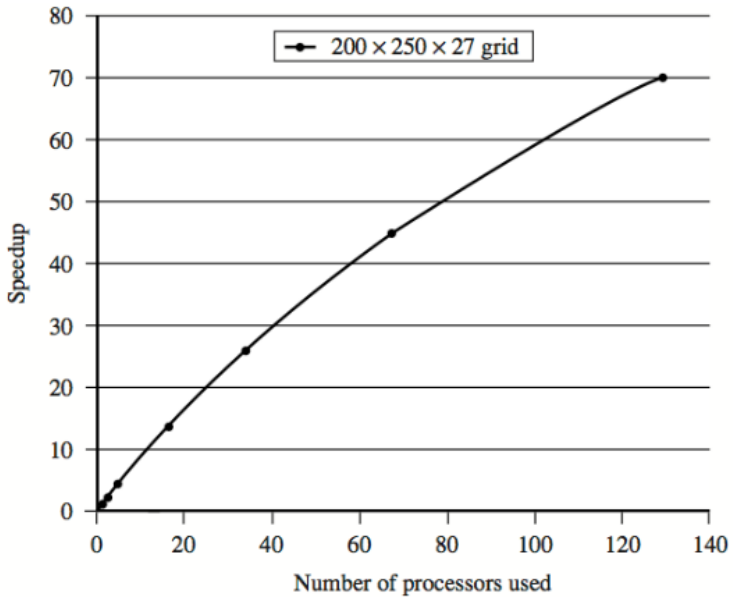
1.6.1 Demonstrating parallel performance

To justify their existence, parallel algorithms must be both correct and efficient. To demonstrate correctness, you validate an algorithm against the serial version to show that it produces the same results across a range of inputs. To demonstrate efficiency, you benchmark an algorithm against the serial version, to show that it is faster.

1.6.2 Speedup

The speedup expected with P processors is equal to the time to solve problem size n with 1 thread divided by the time to solve problem size n with P threads.

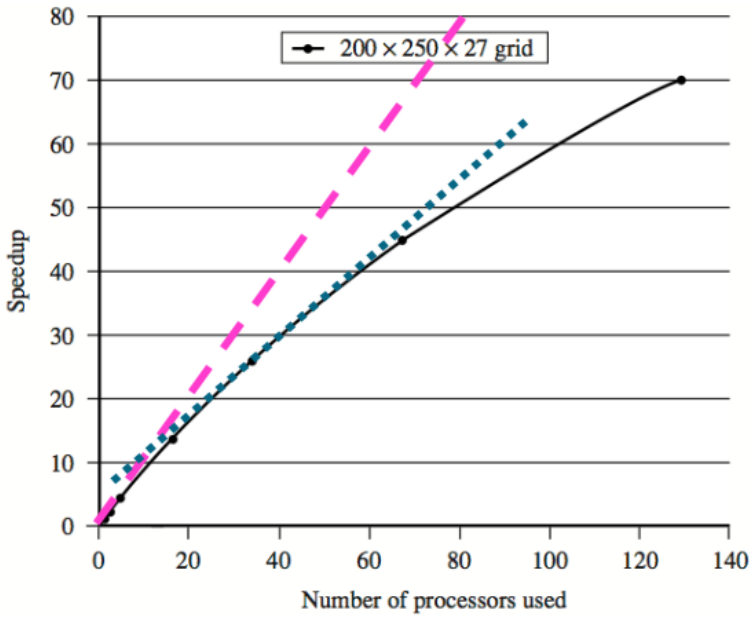
$$S_P = \frac{\text{time to solve } n \text{ with 1 thread}}{\text{time to solve } n \text{ with } P \text{ threads}} = \frac{T_1}{T_P}$$



Note

To compare parallel performance, you need to show speedup graphs and not time graphs!

Perfect linear speedup is when $S_P = P$. In this case, doubling P halves the running time. However, this is hard to achieve in practice. Most algorithms reach an optimal level of performance and then deteriorates.



Superlinear speedup is when $S_P > P$. This is generally not possible in the simple ideal model where the same work is performed and memory effects are ignored. It can occasionally occur in practice due to:

- cache effects / memory hierarchy,
- different algorithmic behavior or
- measurement artifacts

Scalability describes how well performance continues to improve as we increase the processor count and/or the problem size.

1.6.3 Parallel efficiency

Speedup tells us how much faster a program is running. Efficiency tells us how effectively we are using available processors, where perfect efficiency is 100%.

$$E_P = \frac{S_P}{P} = \frac{T_1}{PT_P}$$

As we add processors, speedup may continue to increase, while efficiency decreases. For example,

- Speedup of 6 on 8 cores: $E_8 = 75\%$.
- Speedup of 6 on 32 cores: $E_{32} = 18.75\%$.

1.6.4 Theoretical analysis of expected speedup

There are two key measures of runtime:

- Work (T_1): how long would it take 1 processor. The serial time e.g. for divide-and-conquer, just "sequentialize" the recursive forking.

- Span (T_∞): how long it would take on an ideal machine with unlimited processors. Also called the "critical path length" or "computational depth."

The maximum possible speedup is $T_P = \frac{T_1}{T_\infty}$, i.e. work divided by span. Parallel algorithms are about decreasing span without increasing work too much.

Note

For theoretical work/span analysis, T_1 is the time taken by the parallel algorithm on one processor. For experimental evaluation, speedup is measured against the best appropriate serial implementations:

$$S_P = \frac{T_{\text{serial}}}{T_{\text{parallel},P}}$$

Always state which baseline you use.

1.6.5 Limits on T_P

Work law:

$$\boxed{T_P \geq \frac{T_1}{P}}$$

Each processor executes at most 1 instruction per unit time, and hence P processors can execute at most P instructions per unit time. Thus, to do all the work on P processors, it must take at least T_1/P time.

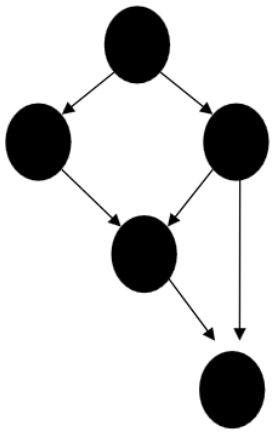
Span law:

$$\boxed{T_P \geq T_\infty}$$

A finite number of processors cannot outperform an infinite number of processors, because the infinite processor machine could just ignore all but P of its processors and mimic a P processors machine exactly.

1.6.6 The DAG (cost graph)

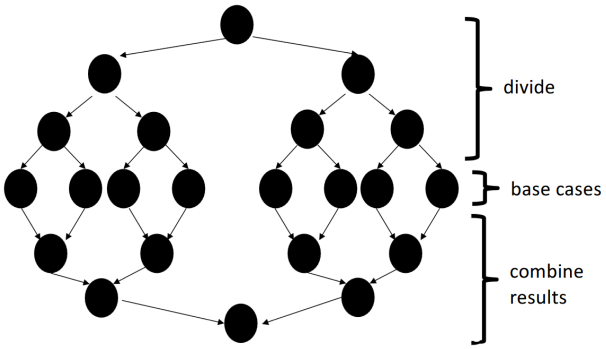
Program execution can be seen as a DAG where nodes are the pieces of work and edges indicate either a fork or a join.



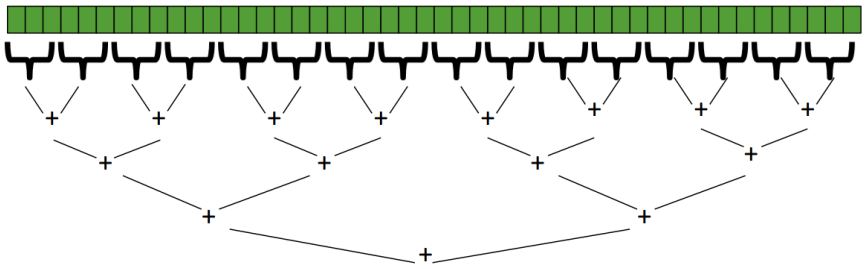
The work (T_1) is the total number of nodes in the DAG. The span (T_∞) is th elongest path in the DAG. The set of instructions form the vertices of the DAG, and the graph edges indicate dependence between instructions. We say that instruction x precedes an instruction y if x must complete before y can begin.

1.6.7 Divide-and-conquer parallel algorithms

fork and **join** are very felxible, but divide-and-conquer maps and reductions use them in a very basic way.



The work is $O(N)$ and the span is $O(\log N)$. Therefore, the maximum speedup is $O(N/\log N)$. As N increasesm the available parallelism $N/\log N$ becomes very large. With enough processors, this algorithm therefore has substantial theoretical parallelism. Anything that can use results from two halves and merge them in $O(1)$ time has the same property.



For well-structured fork/join computations, the framework has expected running time

$$T_P = O(T_1/P) + O(T_\infty)$$

The first term dominates for relatively small P, and the second for sufficiently large P. It's only an "expected" result because internally the framework uses randomness.

1.6.8 Why measured speedup falls below the ideal

Parallelization overhead is the time required to coordinate parallel tasks, as opposed to doing useful work. For example:

- task creation
- task scheduling/allocation

- fork/join operations
- synchronization
- load imbalance

Hadware effects:

- cache misses
- memory bandwidth
- contention

1.6.9 Where does scaling stop?

As the number of processors P increases:

- **Initially:** useful parallel work dominates, i.e. $T_P = T_1/P$. Speedup may be close to linear.
- **Then:** parallel overheads and hardware effects become significant, e.g. task scheduling, joins, load imbalance, cache misses, memory bandwidth.
- **Eventually:** the serial portion / span limits further improvement. Adding processors gives smaller and smaller gains.
- **Beyond this point:** extra processors may actually make the program slower.

1.6.10 Amdahl's law

Amdahl's law is a performance model/law that demonstrates the diminishing returns of adding more processors.

$$S_P = \frac{1}{(1-p) + \frac{p}{P}}$$

Typically, some parts of the program parallelize well, e.g. maps/reductions over arrays. We call this the parallel fraction p. Some parts of the program are inherently sequential, e.g. reading a linked list, getting input, reading from a file, doing computations where each step needs the previous step, etc. We call this the serial fraction $s = 1 - p$. In terms of work and span, this means that the serial portion restricts the minimum span, thus

$$T_\infty \geq (1-p)T_1$$

and so, the maximum possible speedup is:

$$\frac{T_1}{T_\infty} \leq \frac{1}{1-p}$$

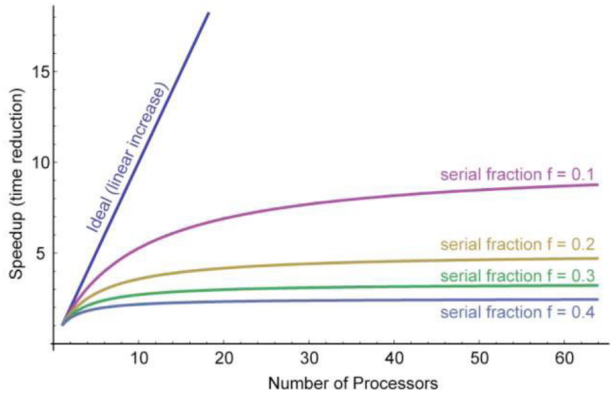
For example, ten processors with 60% parallelizable and 40% sequential. How close are we to a 10-fold speedup?

$$S_{10} = \frac{1}{0.4 + \frac{0.6}{10}} = 2.17$$

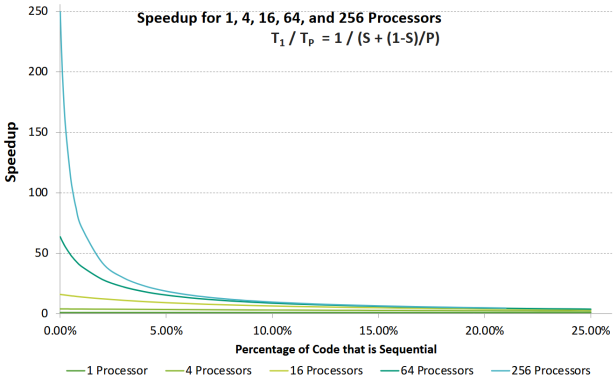
Now, ten processors with 99% parallelizable and 1% sequential. How close are we to a 10-fold speedup now?

$$S_{10} = \frac{1}{0.01 + \frac{0.99}{10}} = 9.17$$

Graphing Amdahl's law:



This is an important graph to consider:



1.6.11 Gustafson's law

Amdahl's law assumes that the total problem size remains fixed as processors are added. In practice, additional computing power is often used to solve larger problems in approximately the same elapsed time. The additional work is often predominantly parallel, so the serial fraction becomes less important as the problem size grows.

1.6.12 Scaling with problem size

Parallel performance depends stringly on the size of the problem.

- A parallel program may perform poorly for small problems because:
  - there is too little useful work to offset the parallel overhead;
  - task creation, scheduling and **join()** costs dominate.
- As the problem becomes larger:

  - there is more useful work available to divide between processors;
  - parallel overhead becomes a smaller fraction of the total runtime;
  - speedup may therefor increase with problem size.

- But eventually:
  - memory bandwidth, cache capacity, memory use or other hardware limitations may become important.

1.6.13 Grid simulations

Increasing the grid dimensions can increase the work in more than one way. For a $R \times C$ simulation, the work per timestep is directly proportional to $R \cdot C$, but the total work is approximately:

$$\boxed{\text{total work} \propto R \times C \times \text{number of timesteps}}$$

If larger grids also require more timesteps to reach convergence, runtime can grow faster than the number of cells alone suggest.

For example:
In an $N \times N$ grid:

- doubling N gives 4 times as many cells;
- if it also takes roughly 2 times as many timesteps;
- total work becomes 8 times larger.

1.6.14 Two kinds of scaling

There are two types of scaling.

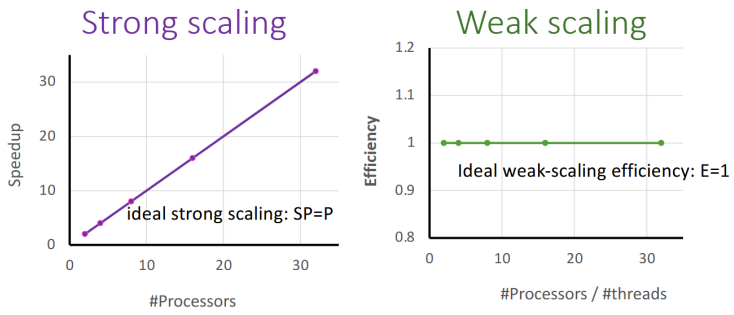
Strong sclaing (Amdahl's law):

- fixed problem size
- defined as how the solution time varies with the number of processors for a fixed total problem size.
- For example, same task with double the number of processors.

Weak scaling (Gustafson's law):

- fixed problem size per processor.
- defined as how the solution time varies with the number of processors for a fixed problem size per processor.
- For example, double the task when you double the number of processors.
- Efficiency:

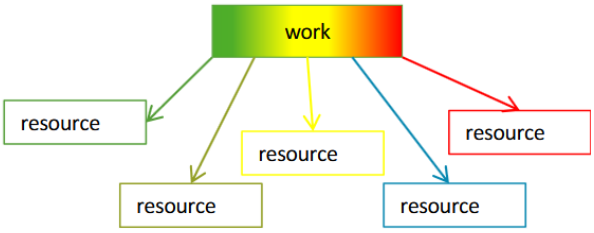
$$E_P = \frac{\text{time to solve problem size } n \text{ with 1 thread}}{\text{time to solve problem size } n \times P \text{ with } P \text{ threads}}$$



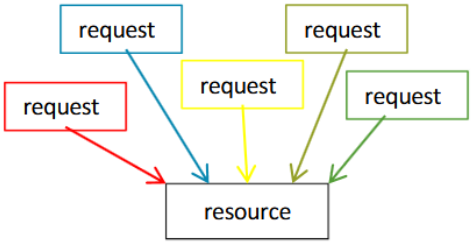
1.7 Concurrency and mutual exclusion

1.7.1 Parallel vs. concurrent programming

So far, we've focused on parallel algorithms which used Fork/Join only. This lowered the span by running the same task on different data, avoiding data races via read-only shared inputs or private array sub-ranges synchronized only at **join()**. This is often called "data decomposition."



Now, we're shifting focus to concurrent programming. Threads are assigned independent tasks and share resources simultaneously to prevent race conditions. This is often called "task decomposition."



Unlike with parallelism, it's not all about implementing algorithms faster:

- **Different tasks** may **need to happen at once** for program to work. For example, animation in a game.
- Simultaneous tasks may be necessary for **responsiveness**. For example, a web server or a GUI.
- Simultaneous tasks may be necessary for **efficient processor utilization**. For example, if 1 thread "goes to disk", have something else to do (often called "latency hiding").

- Simultaneous taks may be necessary for **failure isolation**. Convenient structure is you want to interleave multiple tasks and don't want an exception in one to stop the other.

1.7.2 Synchronization and mutual exclusion

Programmers must use language primitives to enforce event ordering because compilers ignore interleavings. In real life, we typically use a clock to coordinate events. However, synchronization is not timing. There is no guarantee that **Thread.sleep(100)** ensures correctness of the algorithm. synchronization establishes relationships between events rather than relying on how long we expect them to take. **join()** enforces signaling (event A before event B) by waiting for thread termination, whereas mutual exclusion only blocks until a shared resource is free. Competing threads cannot overlap in a critical section (access to shared resources). One thread has access while the others wait until it finishes. Here are some examples of where mutual exlusion is necessary:

- Processing different bank-account operations. What if two threads change the same acount at the same time?
- Using a shared cache (e.g. hash table) of recent files. What if two threads insert the same file at the same time?
- Creating a pipeline with a queue for handing work to next thread in sequence. What if enqueuer and dequeuer adjust a circular array queue at the same time?

1.7.3 Race conditions and atomic classes

Let's look at a simple example of a counter shared between threads.

```
public class Counter {
     private long value;

     Counter() { value=0; }

     public long get() {return value;}

     public void set(long newVal) {
          value=newVal;
     }

     public void incr() {value++;}
}
```

```
public class CounterUpdateThread extends Thread {
        Counter sharedCount;
        int rep;

        CounterUpdateThread(Counter c,int repeats) {
                sharedCount= c;
                rep=repeats;
        }
        public void run() {
                for (int i=0;i<rep;i++) {
                        sharedCount.incr();

                }
        }
}
```

Shared counter object

```
public class TestCounterSafety {
        public static void main(String args[]) throws InterruptedException
{
        int noThrds = 100;
        int addPerThread=100;
        Counter sharedCount= new Counter();

        CounterUpdateThread [] thrds= new CounterUpdateThread[noThrds];
        for (int i=0;i<noThrds;i++) {
                thrds[i]=new CounterUpdateThread(sharedCount,addPerThread);
        }
        for (int i=0;i<noThrds;i++) {
                thrds[i].start();
        }

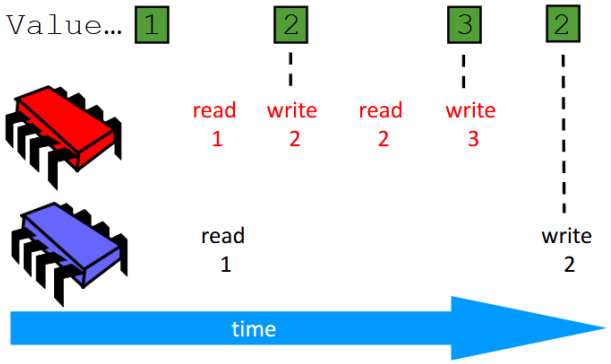
        for (int i=0;i<noThrds;i++) {
                thrds[i].join();
        }

        int expectedVal = noThrds*addPerThread;
        System.out.println("Final value of counter is:"
                + sharedCount.get() + " and should be:" + expectedVal);
        }
}
```

This leads to a race condition. When 100 CounterUpdateThread instances each run **value++** 100 times in TestCounterSafety, the three steps

```
temp = value;
temp = temp + 1;
value = temp;
```

interleave. If two threads read 1, and the first writes 2 then increments to 3, the second thread's delayed write of 2 overwrites 3.



One way we can fix this is with a thread-safe **atomic variable class: java.util.concurrent.atomic**, which contains atomic variable classes for atomic actions on number and objects.

- **AtomicInteger**,
- **AtomicLong**,
- **AtomicBoolean**,
- **AtomicReference**

All of these classes offer: **getAndSet()** and **getAndIncrement()**. Thus, the correct procedure for a thread is:

```
public class Counter {
  private AtomicLong value;  // this is a class!

  Counter() { value=new AtomicLong(0); }

  public long get() { return value.get(); }

  public void set(long val) { value.set(val); }

  public void incr() { value.getAndIncrement(); }

}
```

There are limitations to atomic variables. Atomic variables work when the shared state can be represented by a single atomic variable and each required update can be expressed as one atomic operation. Making individual reads and writes atomic does not automatically make a multi-step operation atomic.

1.7.4 Compound actions and bad interleavings

Take the example of a bank account:

```
class BankAccount {
  private int balance = 0;
  int  getBalance(){ return balance; }
  void setBalance(int x) { balance = x; }

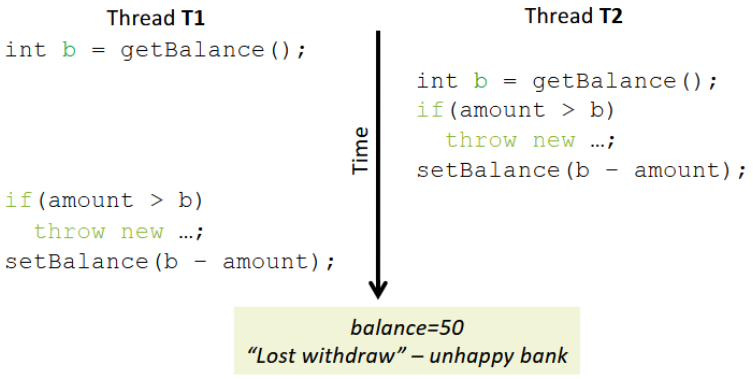
// withdraw is a compound action
  void withdraw(int amount) {
    int b = getBalance();
    if(amount > b)
      throw new WithdrawTooLargeException();
    setBalance(b - amount);
  }
  … // other operations like deposit, etc.
}
```

Why can't we lock only the assignment?

This can lead to bad interleavings. Suppose we have two threads:

- Thread T1 calls **x.withdraw(100)**.
- Thread T2 calls **y.withdraw(100)**.

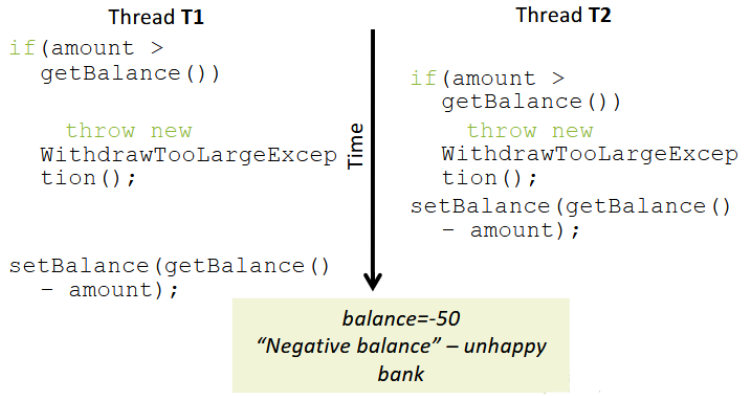
If **x** and **y** refer to different account, then there is no data race. However, if **x** and **y** refer to the same account, then there is a bad interleaving. Assume the account had an initial balance of R150.



It is tempting and almost always wrong to fix a bad interleaving by rearranging or repeating operations, such as:

```
void withdraw(int amount) {
  if(amount > getBalance())
    throw new WithdrawTooLargeException();
  // maybe balance changed
  setBalance(getBalance() – amount);
}
```

Unfortunately, this fixes nothing. It only narrows the problem by one statement. Assume again that we have an initial balance of R150.



We have to make use of a built-in locking mechanism in Java for providing mutual exclusion. It is used by prefixing the method with the keywork **synchronized**. The correct implementation then becomes:

```
class BankAccount {
  private int balance = 0;
  synchronized int getBalance()
    { return balance; }
  synchronized void setBalance(int x)
    { balance = x; }
  synchronized void withdraw(int amount) {
     int b = getBalance();
     if(amount > b)
       throw …
     setBalance(b – amount);
  }
  // deposit would also use synchronized
}
```

1.7.5 Locks, monitors, and reentrancy

Every Java object can act as an intrinsic lock (monitor), whereas primitives cannot. A lock is an abstract data type with operations:

- **new:** make a new lock, initially "not held."
- **acquire:** blocks if this lock is already currently "held." Once "not held," makes lock "held."
- **release:** makes this lock "not held." If one or more thread are blocked on it, exactly one will acquire it.

The lock implementation ensures that upon simultaneous acquires, a correct thing will happen. Explicit **ReentrantLock**'s use **lock()** before a try block and **unlock()** in a finally block so locks are never held forever if an exception is thrown. It tracks the owning thread and hold count (incrementing on re-entry, freeing at 0), preventing deadlock. Here is an example of locks in Java:

```
import java.util.concurrent.locks.Lock;
import java.util.concurrent.locks.ReentrantLock;

private final Lock lk = new ReentrantLock();

void withdraw(int amount) {
   lk.lock();
   try {
      int b = getBalance();
      if (amount > b)
         throw new WithdrawTooLargeException();
      setBalance(b - amount);
   }
   finally {
      lk.unlock();
   }
}
```

There are, however, issues with locks. A lock is a very primitive mechanism. It is still up to you to use it correctly when implementing critical sections. It is also easy to forget to release a lock. An easy fix is to use **synchronized (obj)**. While **synchronized** methods lock the object, **synchronized (obj)** blocks lock only the critical section to reduce contention overhead.

```
lk.lock();
try {
   // critical section
}
finally {
   lk.unlock(); // easy to forget
}
```

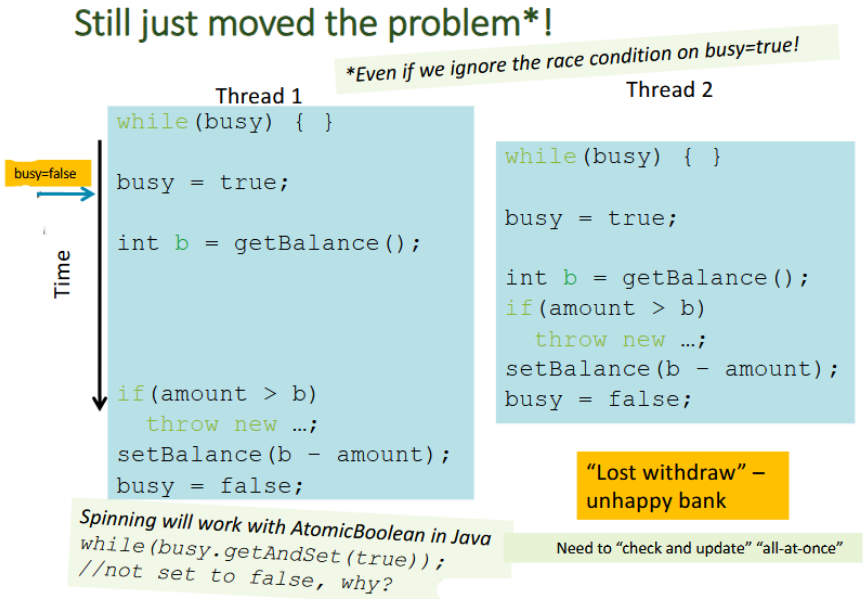
```
synchronized (obj) {
   // automatically released
}
```

Threads can run **synchronized** methods concurrently on different bank accounts, but all methods on the same account (**getBalance()**, **withdraw()**, etc.) must share one lock. In short, locks guarantee mutual exclusion and memory visibility of prior writes to subsequent acquires. Intrinsic locks lack fairness, while **ReentrantLock**'s optionally supports fairness.

1.7.6 Spin-waiting

Writing your own correct mutual-exclusion protocol is much harder than it looks.

```
class BankAccount {
  private int balance = 0;
  private boolean busy = false;
  void withdraw(int amount) {
    while(busy) { /* "spin-wait" */ }
    busy = true;
    int b = getBalance();
    if(amount > b)
      throw new WithdrawTooLargeException();
    setBalance(b – amount);
    busy = false;
  }
  // deposit would spin on same boolean
}
```



For sake of brevity, the bulk of the code for the SharedOven code is omitted. Below are four different attempts at spin-waits.

```
// THIS IS THE CORRECT APPROACH!
public void waitTillFree(String food, int ID) {
    while (busy.getAndSet(true)); // spin
    this.foodItem = food;
    this.chefID = ID;
}
```

```
// INCORRECT
public void waitTillFree(String food, int ID) {
    while (busy.get()); // spin
    busy.set(true);
    this.foodItem = food;
    this.chefID = ID;
}
```

```
// INCORRECT
public void waitTillFree(String food, int ID) {
    if (!busy.get()) {
        this.foodItem = food;
        this.chefID = ID;
    }
}
```

```
// INCORRECT
public void waitTillFree(String food, int ID) {
    while (busy.getAndSet(false)); // spin
    busy.set(true);
    this.foodItem = food;
    this.chefID = ID;
}
```

1.8 Concurrency and thread safety

1.8.1 Thread safety fundamentals

A class can be considered to be thread-safe if it behaves correctly under any runtime thread scheduling or interleaving without requiring external synchronization from calling code, never entering an invalid state. Writing thread-safe code is about managing an object's state. We need to protect data from concurrent access. Race conditions arise when multiple threads access a shared, mutable state and at least one modifies it. All accesses must be coordinated via the **synchronized** keyword, using atomic variables, or locks. There are two kinds of race conditions:

1. **Data races:** simultaneous read/write or write/write of the same memory location.
2. **Bad interleavings:** despite lack of data races, exposing bad intermediate state.

Take the example of the bank account again.

```
class BankAccount {
  private int balance = 0;
  synchronized int getBalance()
    { return balance; }
  synchronized void setBalance(int x)
    { balance = x; }
  synchronized void withdraw(int amount)
  {
     int b = getBalance();
    if(amount > b)
      throw …
    setBalance(b – amount);
  }
  // deposit would also use synchronized
}
```

If you remove the **synchronized** on withdraw, you have a race condition (possible bad interleaving) but not a data race.

1.8.2 Monitors and compound actions

The Java monitor pattern guards all mutable states with one lock, trading concurrency for simplicity by serializing access.

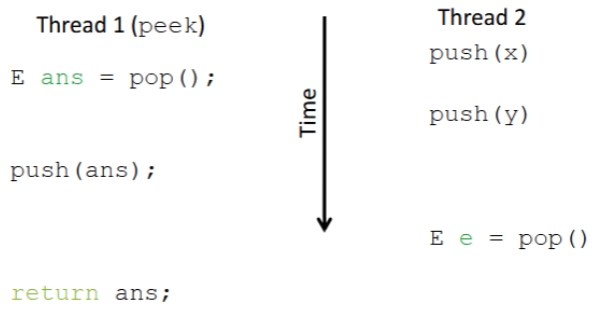
Note on locks

Acquiring the lock associated with an object does not prevent other classes from asseccing the object. It only prevents them from acquiring the same lock.

Compound actions are multi-step operations like *read-modify-write* and *check-then-act* must execute atomically to remain thread-safe. Take the following example of a stack:

```
class Stack<E> {
  … // state used by isEmpty, push, pop
  synchronized boolean isEmpty() { … }
  synchronized void push(E val) { … }
  synchronized E pop() {
    if(isEmpty())
      throw new StackEmptyException();
    …
  }
  E peek() { // this is wrong
     E ans = pop(); //take off list and copy
     push(ans); // put back on list
     return ans;
  }
}
```

Implementing an unsynchronized **peek()** method via a **synchronized pop()** and **synchronized push()** looks like a reader with no net effect. It exposes an inconsistent intermediate state. There are many bad interleaving with the **peek()** method, for example: we want the values returned from **pop()** in LIFO order.



There are two fixes for this.

```
class Stack<E> {
  …
  synchronized E peek(){
     E ans = pop();
     push(ans);
     return ans;
  }
}
```

```
class C {
  <E> E myPeek(Stack<E> s){
    synchronized (s) {
      E ans = s.pop();
      s.push(ans);
      return ans;
    }
  }
}
```

1.8.3 Visibility and volatile variables

Without synchronization on reads and writes, CPU caches, registers, and compiler optimizations can cause threads to see partial or permanently stale values (e.g. linked-list references). Synchronizing all getters and setters on the same lock triggers memory barriers on lock release so any thread subsequently acquiring that lock sees all prior writes. The **volatile** keyword guarantees visibility and ordering (a write happens before subsequent reads), but does not provide mutual exclusion or make the variable atomic. Volatile variables are useful for a simple visibility-based state such as flags, but is not a replacement for locking when compound state changes must be atomic. For example:

```
volatile boolean asleep;
while (!asleep) {
    countSomeSheep();
}
```

1.8.4 Java synchronizers

A synchronizer is any object that coordinates the control flow of threads based on its state, which includes, **join()**, latches, barriers, and blocking queues. Latches act as a gate. No thread can pass until the gate opens, then all can pass. The key difference is that it stays open forever. The **java.util.concurrent.CountDownLatch** allows one or more thread to wait until a set of operations being performed in other threads complete.

```
CountDownLatch latch = new CountDownLatch(3); // countdown from 3
```

- `void await()`

  The calling thread waits until the latch has counted down to zero, unless the thread is interrupted.

- `void countDown()`

  Decrements the count of the latch, releasing all waiting threads if the count reaches zero.

The **java.util.concurrent.CyclicBarrier** allows a fixed number of parties to rendezvous repeatedly at a barrier point. Threads call **await()** when they reach the barrier point and **await()** blocks until all threads have reached the barrier point. Once all threads are there, the barrier is passed, all threads are released and the barrier is reset. CyclicBarriers are useful in programs involving a fixed sized group of threads that must occasionally all wait for each other.

1.8.5 Reordering and memory model

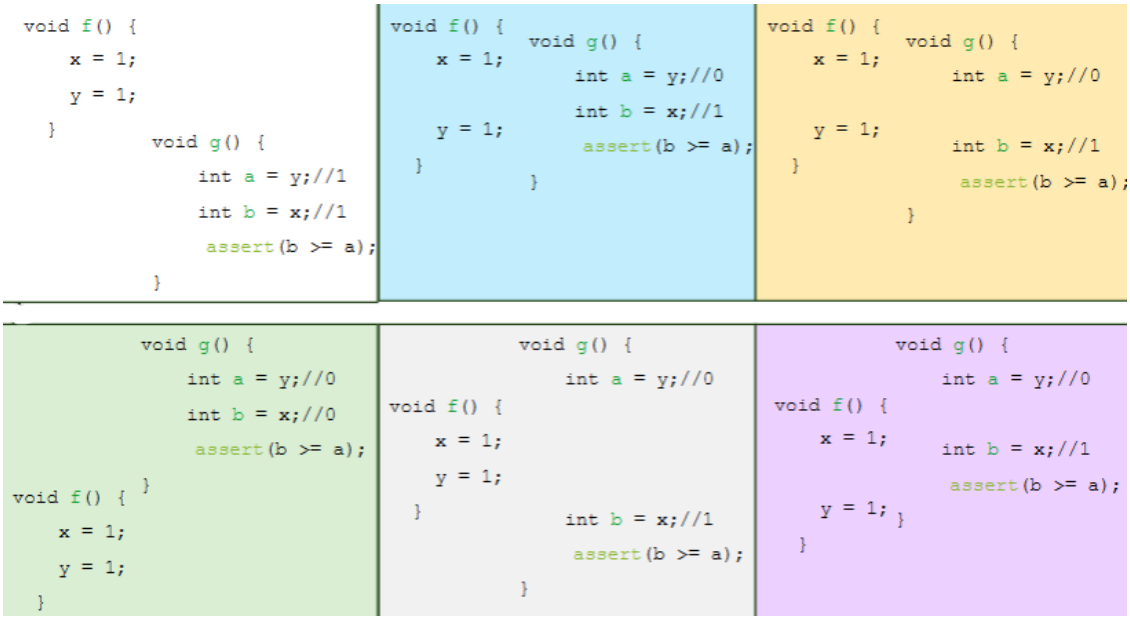
Take the following tricky and surprisingly wrong unsynchronized concurrent code:

```
class C {
  private int x = 0;
  private int y = 0;

  void f() {
    x = 1;
    y = 1;
  }
  void g() {
    int a = y;
    int b = x;
    assert(b >= a);
  }
}
```

There is no interleaving of **f()** and **g()** where the assertion fails.

Proof 1:
Exhaustively consider all possible orderings of access to shared memory (there are 6).



Proof 2:

If **!(b ≥ a)**, then **a==1** and **b==0**. But if **a==1**, then **a=y** happened after **y=1**. And since programs execute in order, **b=x** happened after **a=y** and **x=1** happened before **y=1**. So by transitivity, **b==1**. Contradiction.

However, since both **x** and **y** have data races, the compiler/hardware can reorder independent statements. Thus, the assertion can fail.

1.8.6 The grand compromise

The compiler/hardware will never perform a memory reordering that affects the result of a single-threaded program. It also won't perform a memory reordering that affect the result of a data-race-free multi-threaded program. Correctly synchronized Java programs that are data-race-free behave as though their actions occured in some sequentially consisten interleaving. Fixing the example from before gives us:

```
class C {
  private int x = 0;
  private int y = 0;
  void f() {
    synchronized(this) { x = 1; }
    synchronized(this) { y = 1; }
  }
  void g() {
    int a, b;
    synchronized(this) { a = y; }
    synchronized(this) { b = x; }
    assert(b >= a);
  }
}
```

The Java memory model defines how threads interact through memory. It contains rules for which values may be seen by a read of sahred memory that is updated by multiple threads. As the specification is similar to the memroy models for different architectures, these semantics are known as the Java programming language memory model. The Java memory model requires maintenance of *within thread as-if-serial semantics*. Each thread must have the same result as if executed in serial. The Java memory model defines a partial ordering called **happens-before** on all actions in a program. It guarantees action B sees action A's results only if A *happens-before* B.

- **Monitor lock rule:** a release of a lock happens before every subsequent acquire of the same lock.
- **Volatile variable rule:** a write of a volatile variable happens before every subsequent read of the same volatile variable.

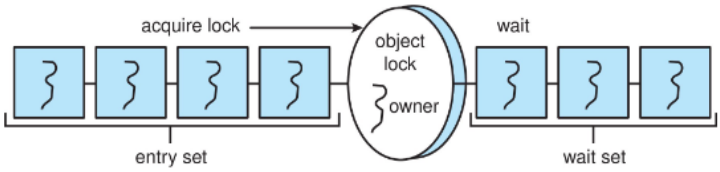
1.9 Condition variables

1.9.1 Busy waiting

There are many synchronization problems when you need a thread to wait until it can continue. The bad approach is to spin. Spinning is when threads are checking the same thing over and over again until they can proceed. This leads to wasted CPU work. Rather use a latch. Condition variables are a mechanism to let threads wait (block) until another thread notifies them to "wake up" and try again to do whatever could not be done before waiting.

1.9.2 Java monitors, sets, thread states

Every Java object has one intrinsic monitor lock acquired on entering a **synchronized** method and released on exit. A thread that tries to acquire an unavailable lock is placed in the object's entry set.



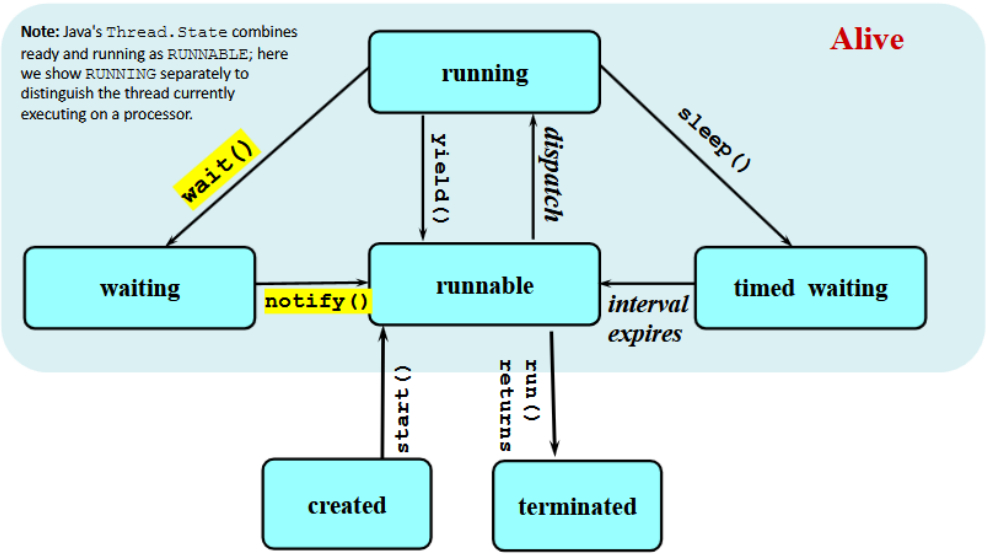
Every object is also a condition variable (it has a wait set). When a thread calls **wait()** while owning the object's monitor:

- It releases the object's monitor (the lock)
- The state of the thread is set to waiting
- The thread is placed in the wait set for the object.

A thread calls **wait()** when it is waiting for a condition to become true. How does a thread get notified? When a thread calls **notify()**:

- An arbitrary thread T is selected from the wait set.
- T is moved from the wait set to the entry set.
- Set the state of T from waiting to runnable.

T can now compete for the lock to check if the condition it was waiting for is now true. Once started, an alive thread has a number of substates:



1.9.3 Condition variable methods and pattern implementations

Java condition variables have three methods that must be called while holding that object's monitor:

- **wait()**
  - Calls to wait should always be inside while-loops that re-check the condition.
  - The call will atomically block the thread, release the lock, and put the thread in the wait set.
  - When the thread is later woken up, it will re-acquire the same lock before wait returns. If other thread also seek the lock, then this may lead to more blocking. There is no guarantee that a notified thread gets the lock next.
- **notify()**
  - wakes up one thread waiting in the object's wait set.
  - If there is no such thread, then notify has no effect, but this is fine.
  - If there are multiple threads blocked, there is no guarantee which one is notified.
- **notifyAll()**
  - wakes up all the threads waiting in the object's wait set.

Otherwise, Java throws **IllegalMonitorStateException**.

1.9.4 Correctness rules and liveness hazards

Problem 1: why while and not if?

```
synchronized (lock) {
    while (!condition) {
        lock.wait();
    }
}
```

After **wait()** returns:

- the wakeup may be spurious (false wakeup is permitted in Java);
- another thread may have changed the state before this thread reacquired the lock;
- a notification merely means "something may have changed," not "your condition is definitely true."

Problem 2: why must condition-check + wait use the same lock?
Notifications are not queued in Java. If a thread isn't already waiting when **notify()** happens, that signal is lost. With **wait()** and **notify()** you should always wait based on a condition (inside a loop). After changing state that may make a waiting condition true, notify the appropriate waiter(s). **notifyAll()** is often safer when multiple threads may be waiting for different conditions on the same monitor, because tvery waiter gets a chance to re-check its condition. It may wake thread unnecessarily, so **notify()** is appropriate when you know exactly one eligible waiter is sufficient.

1.10 Thread liveness

1.10.1 Concurrent correctness properties

Safety properties must always hold true, whereas liveness properties must eventually become true so threads make progress instead of waiting indefinitely. Starvation is the indefinite blocking of a thread, while starvation-free guarantees every thread trying to acquire a resource (such as a lock) eventually succeeds. Deadlock occurs when processes wait forever for an event only a waiting process can cause. Deadlock-free means the system progresses and at least one waiter succeeds.

1.10.2 Deadlocks and deadly embrace

Locks ensure safety, but indiscriminate locking makes programs inherently vulnerable to lock-ordering deadlocks. In general, deadlock forms a dependency-graph cycle where threads $T_1, T_2, \ldots, T_n$ each wait for a resource held by T_{i+1}, and T_n waits for T_1. The simplest two-thread deadlock occurs when Thread A holds lock L while trying to acquire lock M, and Thread B holds lock M while trying to acquire lock L. Java does not automatically recover from deadlocks. Deadlocked threads are permanently disabled, stalling applications or subsystems or degrading performance under worst-case conditions.

1.10.3 Deadlock prevention strategies

Deadlock prevention ensures dependency cycles cannot arise. Never acquiring more than one lock at a time avoids deadlock, though often impractical. Using a single lock across all bank accounts prevents transfer deadlocks, but sacrifices concurrent deposits and withdrawals. The fix is simple. Assign each bank account a unique "account number" and locking the smaller ID first prevents cycles when obeyed program-wide via global analysis. Explicit locks like **ReentrantLock** (unlike **synchronized**) offer **tryLock(timeout, unit)** timeouts, increasing parallelism at the cost of handling alternative paths when acquisition fails.

2 Computer Architecture

2.1 Introduction

2.1.1 What is architecture?

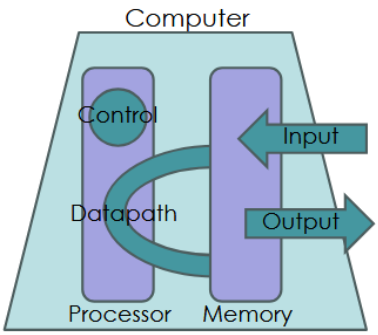
Computer architecture is a study of the overall structure and function of computer systems. In other words, how hardware components link and interact with software.

2.1.2 Assembly programming

A useful lens for understanding computer systems. It exposes the hardware but still (somewhat) device independent. High-level languages often compile first to assembly. It maps one-to-one to machine code instructions.

2.1.3 Components of a computer

- The **processor** gets instructions and data from memory.
- **Input** writes data from an external source to memory, and **output** reads data from memory and transmits to an external source.
- **Control** sends the signals that determine the operations.
- The **datapath** performs the data processing.



2.1.4 Measuring performance

The performance equation:

$$\boxed{\text{Runtime} = \text{IC} \times \text{CPI} \times \text{Clock Cycle Time}}$$

where:

- IC is the instruction count, i.e. the total number of instructions in the program.
- CPI is the number of cycles per instruction, i.e. the average number of clock cycles required for an instruction.
- Clock Cycle Time $= \frac{1}{\text{Clock Rate}}$ is the time for one processor clock period.

This is an approximation of the actual measured runtime, but reasonably accurate.

2.1.5 Performance fallacies

Using a subset of the performance equation is problematic. The clock rate:

$$\text{Clock Rate} = \cancel{\text{IC} \times \text{CPI}} \times \text{Clock Cycle Time}$$

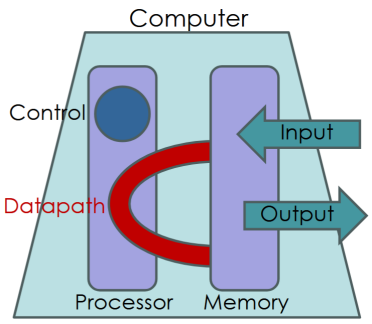
For example, the Intel Core i7-3770K has a clock rate of 3.9GHz. Different architectures may require different numbers of cycles for the same instructions. MIPS = Millions of Instructions Per Second:

$$\text{MIPS} = \cancel{IC} \times \text{CPI} \times \text{Clock Cycle Time}$$

For example, the Intel Core i7-3770K can run 106924 MIPS. Different architectures has different instruction sets, so it cannot be used to directly compare program performance.

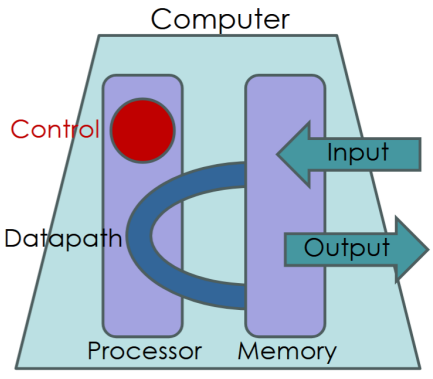
2.2 MIPS

2.2.1 Von Neumann architecture



Datapath:
Instructions are data. Programs can manipulate other programs (e.g. compilers, debuggers, and OS).

Von Neumann bottleneck:
Data transfer takes much longer than data processing and the processor can sit idle. Memory hierarchy is an attempt at mitigation.



Control (Fetch-execute cycle):

- Fetch the next instruction from memory.
- Increment the program counter.
- Decode the instruction.
- Read operands.
- Execute operations.

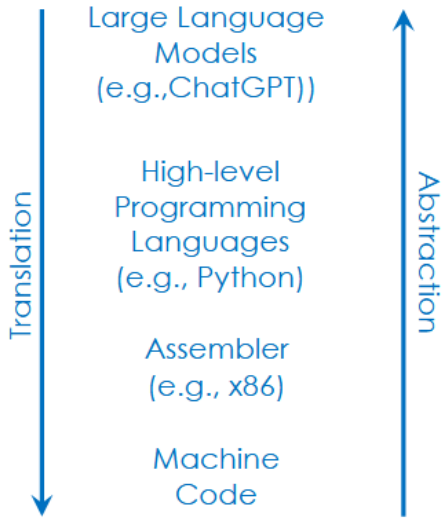
The program counter is a register containing the address of the instruction being executed.

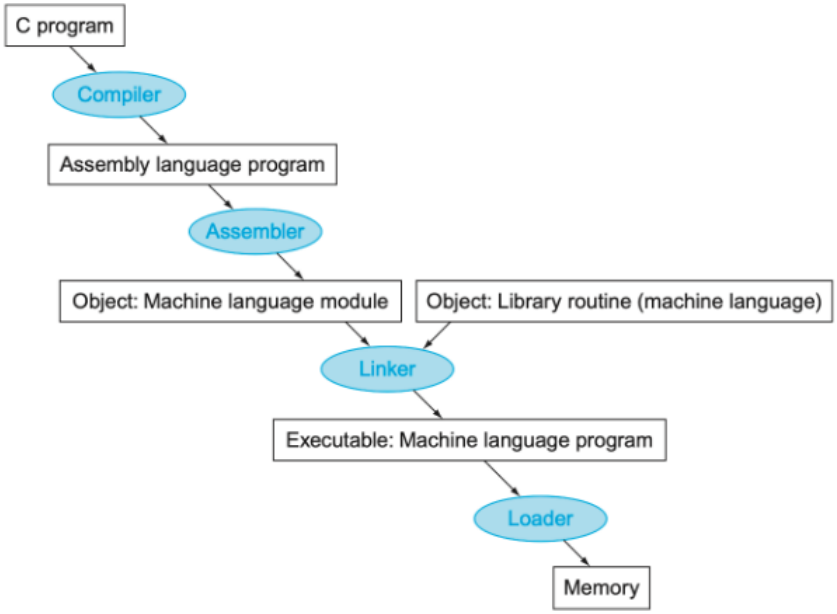
2.2.2 Hierarchy of programming languages

Greater abstraction allows for:

- More concise and natural expression of the solution.
- Greater machine independence.

But performance can still be an issue.





2.2.3 Assembly

Also known as "symbolic machine code." Any low-level programming language with a strong correlation to machine code instructions. There are different variants for each architecture:

- x86
- ARM
- MIPS
- PowerPC

There is usually a one-to-one mapping from assembly to machine code instructions.

MIPS:

```
add     $t0, $t1, $t2
mult    $t3, $t4
```

Machine code (base 16):

```
012a4020
016c0018
```

Machine code (base 2):

```
00000001001010100100000000100000
00000001011011000000000000011000
```

2.2.4 Instruction philosophies

Complex Instructio Set Computing (CISC):

- Larger, more complex instruction set.

- Easier to program, but leads to greater hardware complexity.
- Usually in older architecture, e.g. VAX, x86.

Reduced Instruction Set Computing (RISC):

- Simple instructions, executed rapidly.
- Usually in more modern architectures, e.g. ARM, MIPS.
- Generally, require more instructions for a given algorithm.

Important

There are **no guardrails** in assembly! it can be particularly brutal...

2.2.5 MIPS

MIPS stands for, Microprocessor without Interlocked Pipelined Stages.

- It is a family of RISC architectures.
- It is clean and scalable.
- Used in embedded devices and gaming consoles.
- It has a 32-bit and 64-bit version, but development stopped.

The general instruction format:

```
<operation>     <result> <operand1> <operand2> ...
```

Here is a simple MIPS addition program:

```
.data                       # Declare memory and constants
    num1:   .word   12      # Store 12 at memory location "num1"
    num2:   .word   13      # Store 13 at memory location "num2"
    num3:   .word   0       # Store 0 at memory location "num3"
.text                       # Code section
.globl main                 # Declare the label "main" as global

# Program that loads words from memory and stores words to memory

main:                       # Label "main" (starting point)
    lw      $t1, num1       # Load the value of num1 (12) in $t1
    lw      $t2, num2       # Load the value of num2 (13) in $t2
    add     $t3, $t1, $t2   # $t3 = $t1(12) + $t2(13) = 25
    sw      $t3, num3       # Store the value in $t3 (25) in num3

exit:                       # Label "exit"
    li      $v0, 10         # Load service 10 (terminate program)
    syscall                 # Perform system call with service 10.
```

2.2.6 Assembler directives

Top-level directives: Used to manage code structure

Directive	Description
.text	Indicates that the following items are stored in the user text segemtn, typically instructions.
.data	Indicates that the following data items are stored in the data segment.
.globl sym	Declare the symbol "sym" is global and can be referenced from other files.

Data directives:
On 32-bit architecture:

- byte = 8 bits
- halfword = 2 bytes
- word = 4 bytes

Directive	Description
.word w1,...,wn	Store n 4-byte quantities in successive memory locations.
.half h1,...,hn	Store n 2-byte quantities in successive memory locations.
.byte b1,...,bn	Store n 1-byte quantities in successive memory locations.
.ascii str	Store a **non**-null-terminated string in memory.
.asciiz str	Store a null-terminated string in memory.
.float f1,...,fn	Store n floating point single precision numbers in successive memory locations.
.double d1,...,dn	Store n floating point double precision numbers in successive memory locations.
.space n	Reserve n successive bytes of memory.

2.2.7 Labels

The notation of variables is a high-level programming concept not available in MIPS assembly. Instead, labels are used to identify addresses in data and code, e.g. "main" in code and "num1" in data. Importantly, RISC arithmetic and logic instructions do not operate on data memory directly but on registers instead. So, transfer from data memory to registers is a must!

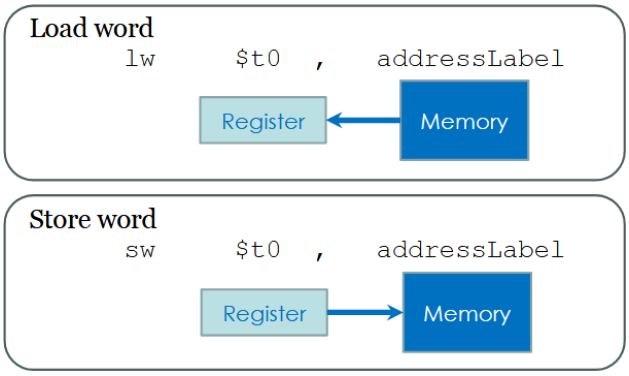
2.2.8 Registers

Processors contain a limited number of high-speed memory chips, called registers. MIPS32 has 32 general purpose and 32 floating point registers, each 32 bits wide. It is capable of addressing $2^{32} = 4,294,967,296 = 4$GB of RAM. Unlike 64-bit which can address ∼18.1EB (exabytes).

Number	Name	Comments
$0	$zero, $r0	Built into hardware to ALWAYS be zero.
$1	$at	Reserved for assembler.
$2, $3	$v0, $v1	First and second return values, respectively. Used for syscalls.
$4, ..., $7	$a0, ..., $a3	First four arguments to functions. Often used with syscalls.
$8, ..., $15	$t0, ..., $t7	Temporary registers.
$16, ..., $23	$s0, ..., $s7	Saved registers (must be restored after function calls).
$24, $25	$t8, $t9	More temporary registers.
$26, $27	$k0, $k1	Reserved for kernel (operating system).
$28	$gp	Global pointer.
$29	$sp	Stack pointer.
$30	$fp	Frame pointer.
$31	$ra	Return address.
	$f0, ..., $f32	Single precision floating point registers.

2.2.9 Memory

Operations can only be performed on register, thus we need to move values from memory to registers before use.



Memory structure:
Logically, a 1D array of bytes. Labels map to addresses in memory. Less significant bits placed at lower addresses (little endian).

For example, $t0 holds number 0x65676361. $s0 holds address 12. Then,

```
sw      $t0, $s0
```

result in the following:

Address	Value
19	
18	
17	
16	
15	65
14	67
13	63
12	61
11	
10	
9	
8	
7	
6	
5	
4	
3	
2	
1	
0	

Each new label in *.data* is aligned on type-specific boundary. For example, word addresses are multiples of 4. But, comma separated values are packed as tightly as possible.

Memory access with offsets:
We need offset instructions to address elements in a sequence of numbers. For example:

```
arr:    .word 11, 12, 13
```

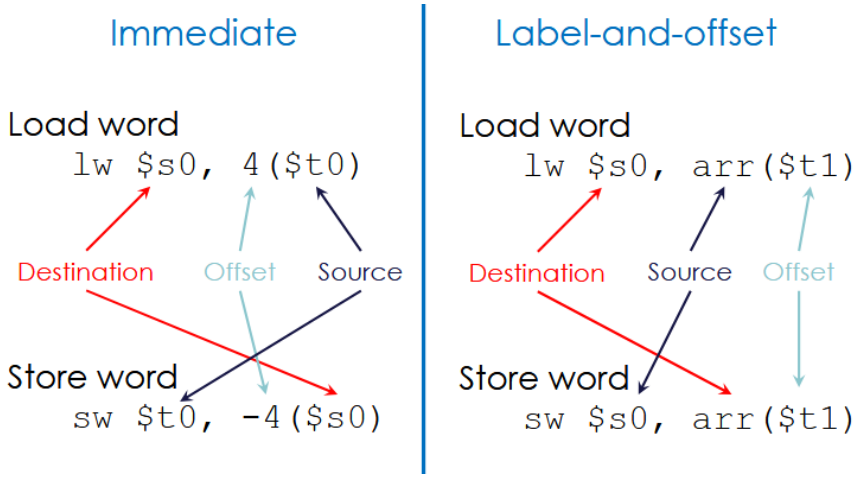
Here *arr* is the address of the first element. Much like indexing an array. We can use byte offsets to address elements.

Immediate form:

```
lw      $s0, 4($t0) # $t0 holds address of arr
```

Label and offset form:

```
lw      $s0, arr($t1) # if $t1 holds 4
```

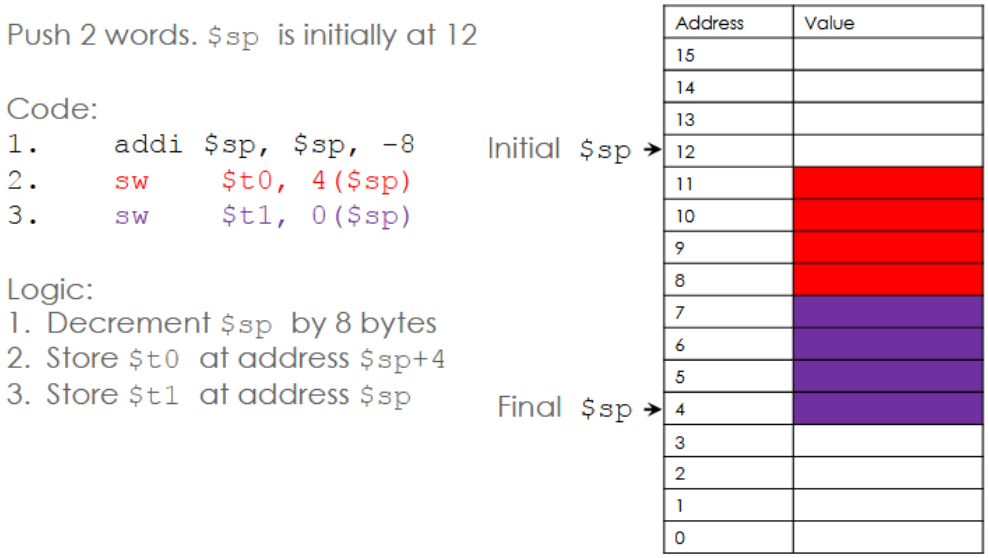


2.2.10 Stack

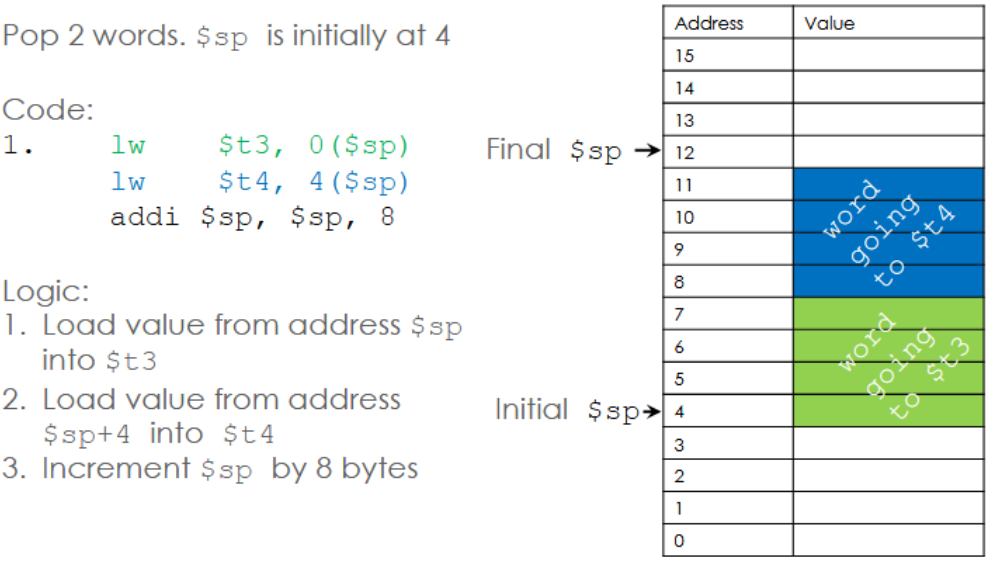
We often need to free up registers but retain their values. We can use stack memory:

- Push registers onto the stack.
- Pop values from the stack back into registers.
- Needs to be managed by hand using the register *$sp* pointing to the end of the stack.
- Stack grows from top downward.

Stack Push:



Stack Pop:



2.2.11 Data transfer instructions

Instruction	Example
Load word	lw $1, 100($2)
Store word	sw $1, 100($2)
Load upper immediate	lui $1, 100
Load address	la $1, label
Load immediate	li $1, 100
Move from hi	mfhi $2
Move from lo	mflo $2
Move	move $1, $2

3 Theory of Computation

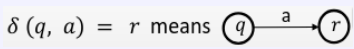
3.1 Regular Languages

3.1.1 Finite automata

Definition: Finite automata

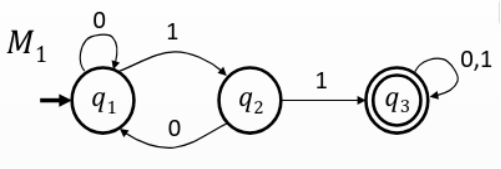
A **finite automaton** is a 5-tuple $(Q, \Sigma, \delta, q_0, F)$ where:

- Q is a finite set of all possible states.
- Σ is a finite set of all alphabet symbols.
- δ is a transition function $\delta : Q \times \Sigma \to Q$. In other words,



- q_0 is the start state.
- F is the set of accept states.

Example:
Let's consider the following finite automaton.



Here, $M_1 = (Q, \Sigma, \delta, q_1, F)$ is a finite automaton. For sake of example, here are the exact details of M_1.

- $Q = \{q_1, q_2 . q_3\}$
- $\Sigma = \{0, 1\}$

- $F = \{q_3\}$

It accepts a finite string as input. The output is either *ACCEPT* or *REJECT*. The computation goes as follows:

1. Begin at start state.
2. Read input symbols.
3. Follow corresponding transitions.
4. *ACCEPT* if we reach the accept state, otherwise *REJECT*.

For example: 01101 $\rightarrow$ *ACCEPT*, while 00101 $\rightarrow$ *REJECT*. Here, M_1 accepts all string in A where $A = \{w|w \text{ contains substring } 11\}$. We say that A **is the language of** $M_1 \iff M_1$ **recognizes** $A \iff A = L(M_1)$.

3.1.2 Strings and languages

A **string** is a finite sequence of symbols $w_1 w_2 \cdots w_n \in \Sigma$. A **language** is a set of string (finite or infinite). The **empty string** ε is the string of length 0. The **empty language** $\emptyset$ is the set with no strings.

Definition: Accepting strings

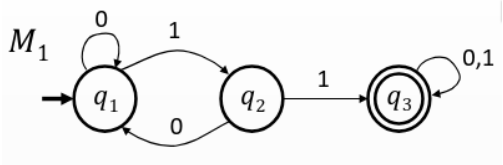
A finite automaton M **accepts string** $w = w_1 w_2 \cdots w_n$ for each $w_i \in \Sigma$ if there exists a sequence of states $r_0, r_1, \ldots, r_n \in Q$ where:

- $r_0 = q_0$
- $r_i = \delta(r_{i-1}, w_i)$ for $1 \leq i \leq n$
- $r_n \in F$

Definition: Regular language

We say a language is **regular** if some finite automaton recognizes it.

Example 1:



$L = (M_1) = \{w|w \text{ contains substring } 11\} = A$. Therefore, A is regular.

Example 2:
Let $B = \{w|w \text{ has an even number of 1s}\}$. In this case, B is regular. (not shown)

Example 3:
Let $C = \{w|w \text{ has equal number of 0s and 1s}\}$. In this case, C is not regular. (provable)

3.1.3 Regular expressions

Definition: Regular operations

Let A, B be languages.

- **Union:** $A \cup B = \{w | w \in A \text{ or } w \in B\}$.
- **Concatenation:** $A \circ B = \{xy | x \in A \text{ and } y \in B\} = AB$.
- **Star:** $A^* = \{x_1 \dots x_k | x_i \in A \text{ for } k \geq 0\}$. (Note: $\varepsilon \in A^*$)

Regular expressions are built from $\Sigma, \emptyset, \varepsilon$ by using regular operations $\cup, \circ, *$.

Example 1:

- $(0 \cup 1)^* = \Sigma^*$ gives all strings over Σ.
- Σ^*1 gives all strings that end with 1.
- $\Sigma^*11\Sigma^*$ gives all strings that contain $11 = L(M_1)$.

Example 2:
Let $A = \{\text{good, bad}\}$ and $B = \{\text{boy, girl}\}$.

- $A \cup B = \{\text{good, bad, boy, girl}\}$.
- $A \circ B = AB = \{\text{goodboy, goodgirl, badboy, badgirl}\}$.
- $A^* = \{\varepsilon, \text{good, bad, goodgood, goodbad, badgood, badbad, goodgoodgood, } \dots\}$

3.1.4 Nondeterministic finite automata

Definition: Nondeterministic finite automata

A **nondeterministic finite automata (NFA)** N is a 5-tuple $(Q, \Sigma, \delta, q_0, F)$ where everything is the exact same as a DFA except δ. Here, $\delta : Q \times \Sigma_\varepsilon \to \mathcal{P}(Q) = \{R \mid R \subseteq Q\}$.

New features of nondeterminism:

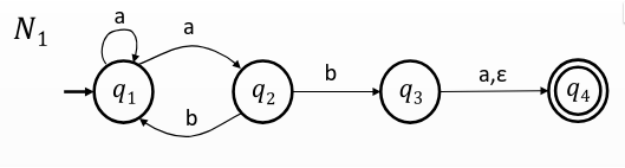
- Multiple paths possible (0, 1, or many at each step)
- ε-transition is a "free" move without reading input.
- Accept input if **some** path leads to the accept state.

Nondeterminism doesn't correspond to a physical machine we can build. However, it if useful mathematically. Here are some ways to think about it:

- **Computationally:** Fork new parallel thread and accept if any thread leads to an accept state.
- **Mathematically:** Tree with branches. Accept if any branch leads to an accept state.

- **Magical:** Guess at each nondeterministic step which way to go. The machine always makes the right guess that leads to the accept state, if possible.

Example:



Here, we have $\delta(q_1, a) = \{q_1, q_2\}$, but $\delta(q_1, b) = \emptyset$.

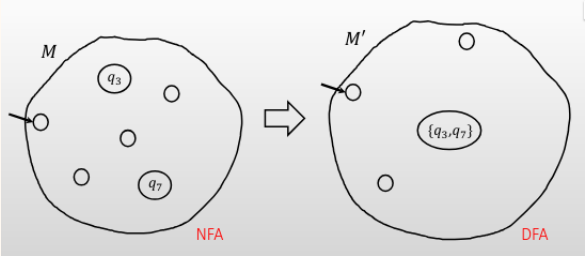
3.1.5 Converting NFAs to DFAs

Theorem

If an NFA recognizes A, then A is regular.

Proof:
Let NFA $N = (Q, \Sigma, \delta, q_0, F)$ recognize A. We need to construct a DFA $M = (Q', \Sigma, \delta', q_0', F')$, ignoring the ε-transitions. Here M is defined as:

- $Q' = \mathcal{P}(Q)$.
- $\delta'(R \in Q', a) = q \in \delta(r, a)$ for some $r \in R$.
- $q_0' = \{q_0\}$.
- $F' = \{R \in Q' | R \cap F\}$.



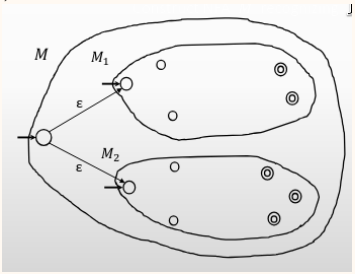
3.1.6 Closure properties for regular languages

Theorem: Closure under $\cup$.

If A_1 and A_2 are regular languages, then so is $A_1 \cup A_2$.

Proof:
Let $M_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ recognize A_1 and similarly, let $M_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$ recognize A_2. We need to construct $M = (Q, \Sigma, \delta, q_0, F)$ where

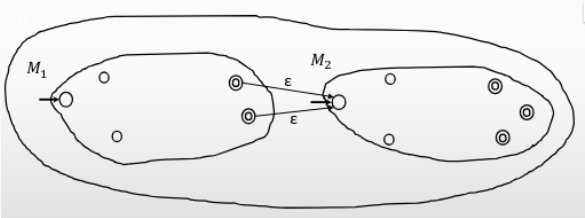
- $Q = Q_1 \times Q_2 = \{(q_1, q_2) \,|\, q_1 \in Q_1 \wedge q_2 \in Q_2\}$.
- $q_0 = (q_1, q_2)$.
- $\delta((q, r), a) = \big(\delta_1(q, a), \delta_2(q, a)\big)$.
- $F = (F_1 \times Q_2) \cup (Q_1 \times F_2)$.



Theorem: Closure under $\circ$.

If A_1 and A_2 are regular languages, then so is A_1A_2.

Proof:
Let $M_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ recognize A_1 and similarly, let $M_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$ recognize A_2. We need to construct $M = (Q, \Sigma, \delta, q_0, F)$ where

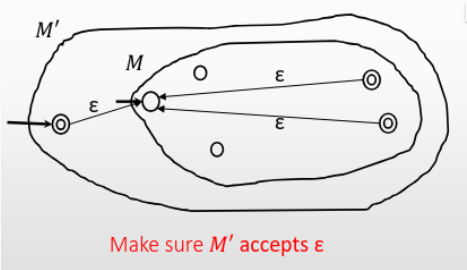


M should accept input w if $w = xy$ where M_1 accepts x and M_2 accepts y. The nondeterministic M has the option to jump to M_2 when M_1 accepts.

Theorem: Closure under $*$

If A is a regular language, then so is A^*.

Proof:
Given a DFA M recognizing A, construct an NFA M' recognizing A^*.



M' should accept input w if $w = x_1x_2 \ldots x_k$ where $k \geq 0$ and M accepts each x_i.

3.1.7 Converting a regular expression to NFA

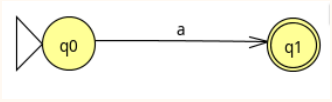
Theorem

If R is a regular expression and $A = L(R)$, then A is regular.

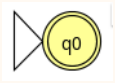
Proof:
Convert R to the equivalent NFA M.

If R is atomic, an equivalent M is:

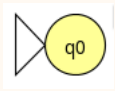
- $R = a$ for $a \in \Sigma$



- $R = \varepsilon$



- $R = \emptyset$

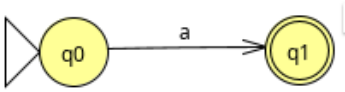


If R is composite:

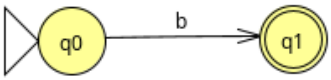
- $R = R_1 \cup R_2$
- $R = R_1 \circ R_2$
- $R = R_1^*$

Example:
Convert $(a \cup ab)^*$ to an equivalent NFA.

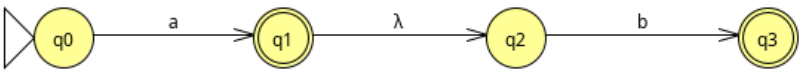
- a:



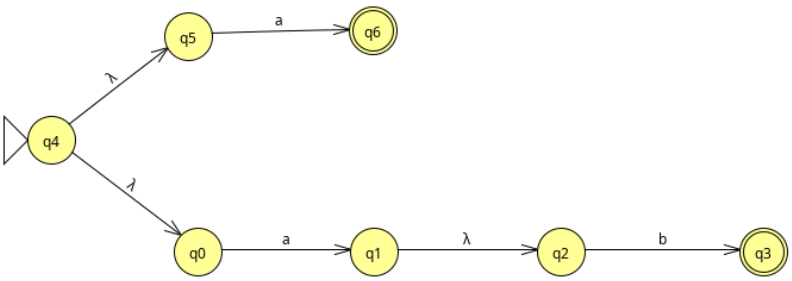
- b:



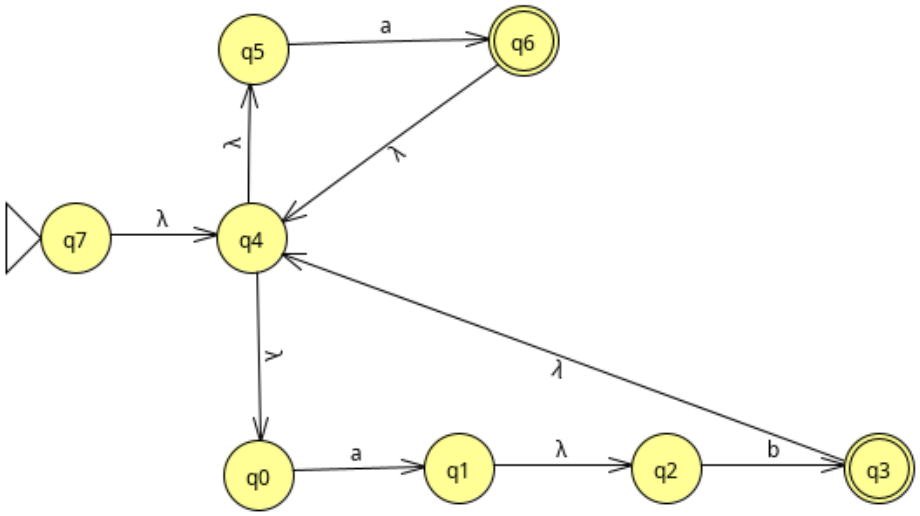
- ab:



- $a \cup ab$:



- $(a \cup ab)^*$:



Reminder

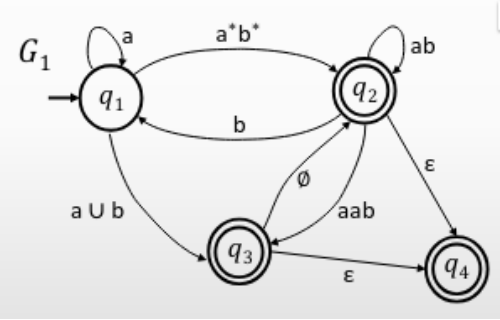
A λ transition is equivalent to an ε transition.

3.1.8 Generalized NFA

Definition: Generalized NFA

A **generalized nondeterministic finite automaton (GNFA)** is similar to an NFA, but allows regular expressions as transition labels.

Example:



For convenience, we will assume:

- One accept state, separate from the start state.
- One arrow from each state to each state, except
  - only exiting the start state
  - only entering the accept sate

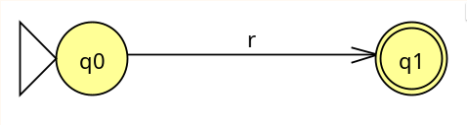
We can easily modify a GNFA to have this special form.

3.1.9 Converting a GNFA to a regular expression

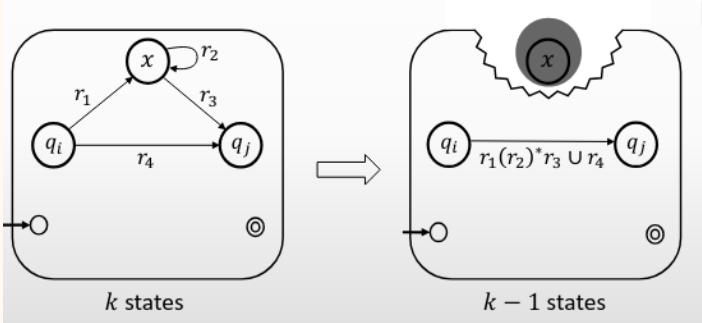
Lemma

Every GNFA G has an equivalent regular expressions R.

Proof:
We will make use of an inductive argument on the number of states k of G. For the base step ($k = 2$):



Assume the Lemma holds true for $k - 1$ states. Now we want to prove it holds for k states. The central idea is to convert a k-state GNFA to an equivalent $(k - 1)$-state GNFA.



1. Pick any state x except the start and accept states.
2. Remove x.
3. Repair the damage by recovering all paths that went through x.
4. Make the indicated change for each pair of states q_i, q_j.

Thus DFAs and regular expressions are equivalent. ■

3.1.10 Non-regular languages

How do we show a language is not regular?

- Remember, to show a language is *regular*, we give an equivalent DFA.
- To show a language is *not regular*, we must give a proof.
- It is not enough to say you couldn't find a DFA for it, therefore it isn't regular!

Example 1:
Let $B = \{w | w$ has equal numbers of 0s and 1s$\}$. One's intuition syas that B is not regular since DFAs cannot count unboundedly.

Example 2:
Let $C = \{w | w$ has equal numbers of 01 and 10 substrings$\}$. As examples, $0101 \notin C$ and $0110 \in C$. Again, one's intuition says that C is not regular since DFA cannot count unboundedly. However, C is regular!

3.1.11 Proving non-regularity

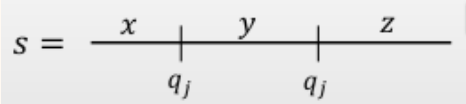
Pumping Lemma

For every regular language A, there exists a number p, called the "pumping length", such that if $s \in A$ and $|s| \geq p$ then $s = xyz$ where

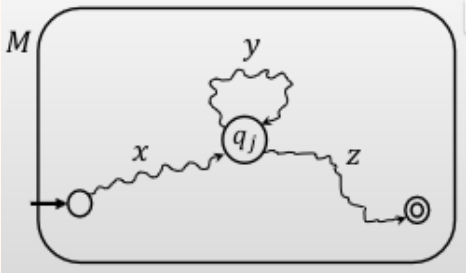
1. $xy^iz \in A$ for all $i \geq 0$ $\quad (y^i = \underbrace{yy \cdots y}_{i})$.
2. $y \neq \varepsilon$.
3. $|xy| \leq p$.

Informally, if A is regular, then every long string in A can be "pumped" and the result stays in A.

Proof:
Let DFA M recognize A. Let p be the number of states in M. Pick $s \in A$ where $|s| \geq p$.



M will repeat a state q_j when reading s because s is so long. The path that M follows when reading s.



Exmaple 1:
Let $D = \{0^k1^k | k \geq 0\}$. Show that D is not regular.

Assume, for the sake of contradiction, that D is regular.
Since D is regular, the pumping lemma must hold. Let $s = 0^p1^p \in D$.
The pumping lemma says that we can divide $s = xyz$.
But $xyyz$ has excess 0s, and thus $xyyz \notin D$ contradicting the pumping lemma.
Therefore, our assumption is false and thus D is not regular.

Example 2:
Let $F = \{ww | w \in \Sigma^*\}$. Say $\Sigma^* = \{0, 1\}$. Show that F is not regular.

Assume, for the sake of contradiction, that F is regular.
Since F is regular, the pumping lemma must hold.
We need to choose $s \in F$. Let $s = 0^p 10^p 1 \in F$.
We can show that $s = xyz$ cannot be pumped satisfying the 3 conditions.
Thus, $xyyz \notin F$, which is a contradiction. Therefore, F is not regular.

Example 3:
Let $B = \{w | w$ has equal numbers of 0s and 1s$\}$. Show that B is not regular.

Assume, for the sake of contradiction, that B is regular.
We know 0^*1^* is regular, and so $B \cap 0^*1^*$ is regular (closure under intersection).
But $D = B \cap 0^*1^*$ and we already showed D is not regular (from example 1).
Therefore, B is not regular.

Example 4:
Let $A = \{1^{n^2} | n \geq 0\}$. *(A contains all strings of 1s whose length is a perfect square).* Show that A is not regular.

Assume, for the sake of contradiction, that A is regular.
Let p be the pumping length given by the pumping lemma. Take $s = 1^{p^2} \in A$.
The pumping lemma guarantees $s = xyz$ with $s = xy^iz \in A$ for some $i \geq 0$.
Consider the two strings xyz and xy^2z. The length of these strings differ by y.
By contition 3, $|xy| \leq p$ and thus $|y| \leq p$. Then, we have $|xyz| = p^2$ and so $|xy^2z| \leq p^2 + p$.
But $p^2 + p < p^2 + 2p + 1 = (p + 1)^2$.
Moreover, condition 2 implies that y is not the empty string and so $|xy^2z| > p^2$.
Therefore, the length of xy^2z lies strictly between the consecutive squares p^2 and $(p + 1)^2$.
Hence this length cannot be a perfect square itself, which is a contradiction.
Thus, $xy^2z \notin A$ and so we conclude that A is not regular.

Example 5:
Let $E = \{0^i 1^j | i > j\}$. Show that E is not regular.

Assume, for the sake of contradiction, that E is regular.
Let p be the pumping length given by the pumping lemma. Take $s = 0^{p+1}1^p \in E$.
By condition 1, $xy^iz \in E$ even when $i = 0$. Consider the string $xy^0z = xz$.
Removing string y decreases the number of 0s in s.
But, recall that s has just one more 0 than 1.
Therefore, xz cannot have more 0s than 1s and so $s = 0^{p+1}1^p \notin E$.
Hence a contradiction. Therefore, E is not regular.

3.2 Context-free languages

3.2.1 Context-free grammars

Definition: Context-free grammar

A **Context Free Grammar (CFG)** G is a 4-tuple (V, Σ, R, S) where

- V is the finite set of variables.
- Σ is the finite set of terminal symbols.
- R is the finite set of rules (rule form: $V \to (V \cup \Sigma)^*$).
- S the starting variable.

For $u, v \in (V \cup \Sigma)^*$ write

1. $u \Rightarrow v$ if you can go from u to v with one substitution step in G.
2. $u \overset{*}{\Rightarrow} v$ if you can go from u to v with some number of substitution steps in G. $u \Rightarrow u_1 \Rightarrow u_2 \Rightarrow \cdots \Rightarrow u_k \Rightarrow v$ is called a *derivation* of v from u. Furthermore, if $u = S$ then it is a *derivation* of v.

$L(G) = \{w | w \in \Sigma^* \text{ and } S \overset{*}{\Rightarrow} w\}$.

Definition: Context-free language

A is a **Context Free Language (CFL)** is $A = L(G)$ for some CFG G.

3.2.2 Ambiguity

Take the following two CFGs:

G_1

$E \to E + T \mid T$

$T \to T \times F \mid F$

$F \to (E) \mid a$

G_2

$E \to E + E \mid E \times E \mid (E) \mid a$

Both G_1 and G_2 recognize the same language, i.e. $L(G_1) = L(G_2)$. However, G_1 is an unambiguous CFG and G_2 is ambiguous.

3.2.3 Pushdown automaton

Definition: Pushdown automaton

A **Pushdown Automaton (PDA)** is a 6-tuple $(Q, \Sigma, \Gamma, \delta, q_0, F)$:

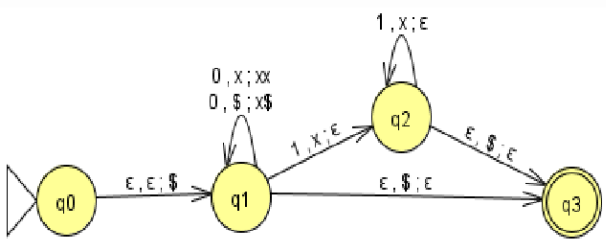
- Σ is the input alphabet.
- Γ is the stack alphabet.
- $\delta : Q \times \Sigma_\varepsilon \times \Gamma_\varepsilon \to \mathcal{P}(Q \times \Gamma_\varepsilon)$
  $\delta(q, a, c) = \{(r_1, d), (r_2, e)\}$

We accept if some thread is in the accept state at the end of the input string.

A PDA operates like an NFA except can write/add (push) or read/remove (pop) symbols from the top of the stack. The input appears on a tape.

Example 1:
PDA for $D = \{0^k 1^k : k \geq 0\}$

1. Read 0s from input, push 0 onto the stack until a 1 is read.
2. Read 1s from input, while popping 0s from the stack.
3. Enter the accept state if the stack is empty.



How to read transitions for PDAs

Example 1: Take the transition ε, ε; \$.

- The first ε is the input read from the tape. Since we're reading ε, this is equivalent to a blank on the tape.
- The second ε is the value on the top of the stack. Since we're reading ε, we assume the stack is empty.
- The last \$ is part of the stack alphabet. This signifies the bottom of the stack. We want to "push" this value onto the stack.

Example 2: Take the transition ε, \$; ε.

- The first ε is the input read from the tape. Since we're reading ε, this is equivalent to a blank on the tape.
- The \$ is the value we expect on the top of the stack.

- The second ε is the value we push onto the stack. We're effectively replacing the bottom-of-stack character with a blank, clearing the stack.

Example 2:
PDA for $B = \{ww^{\mathcal{R}} : w \in \{0,1\}^*\}$ with sample input: 011110.

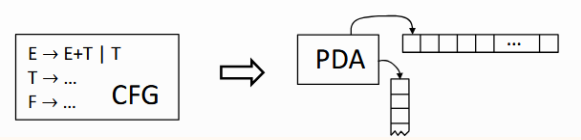
1. Read and push input symbols onto the stack. Non-deterministically either repeat or go to (2).
2. Read input symbols and pop stack symbols, and compare. If they are ever not equal, then the thread rejects.
3. Enter the accept state if the stack is empty.

3.2.4 Converting CFGs to PDAs

Theorem

If A is a CFG, then some PDA recognizes A.

Proof:
Convert A's CFG to a PDA.



1. Push the start symbol onto the stack.
2. If the top of the stack is:
   - *Variable:* replace with right hand side of rule (non-deterministic choice).
   - *Terminal:* pop it and match with next input symbol.
3. If the stack is empty, enter the accept state.

3.2.5 Equivalence of CFGs and PDAs

Theorem

A is a CFL if and only if some PDA recognizes A. *(Proof not examinable)*

Corollaries

1. Every regular language is a CFL.
2. If A is a CFL and B is regular, then $A \cap B$ is a CFL.

Note 1: If A and B are CFLs then $A \cap B$ may not be a CFL. Therefore, the class of CFLs is not closed under $\cap$.

Note 2: The class of CFL is closed under $\cup, \circ, *$.

3.2.6 Chomsky normal form (CNF)

Chomsky normal form is a simplification of grammars:

- If the start symbol is recursive, remove it.
- Removing ε-productions.
- Removing unit productions.
- Removing "useless" productions.

Converting to CNF

Given a grammar $G = \{V, T, S, P\}$, we can derive $G' = \{V, T, S', P'\}$ as follows:

Removing recursive start symbol:
Introduce a new start symbol S'. Let P' be the production rule set P of G with the following rule added to it: $S' \to S$.

Remove ε-productions:
If there is a production of the form $A \to \varepsilon$ then A is a nullable variable. If $B \to A_1 A_2 \cdots A_n$ and $A_1 A_2 \cdots A_n$ are nullable variables, then B is also a nullable variable. A given grammar G can be converted into a grammar G' that does not contain nullable variables.

1. Remove each productions rule of the form $A \to \varepsilon$.
2. Add each rule of the form $A \to s_1 s_2 \cdots s_n$ with $s_i \in V \cup T$ such that each s_i is not a nullable variable to P'.
3. For each production rule $A \to s_1 N_1 s_2 N_2 \cdots s_n N_n$, where each s_i is not a nullable variable and $N_1, N_2, \ldots, N_n$ are nullable variables, add a copy of the rule with one or more N_i's replaced by ε to P', e.g. $A \to s_1 s_2 N_2 \cdots s_n N_n$. Note that if the s_i's are also nullable, then we do not add $A \to \varepsilon$ unless A is the start symbol.

Remove unit productions:

- Draw a dependency graph with edge (A, B) for each unit production of the form $A \to B$.
- From the graph, identify all A B.
- For all A and B identified, add a production of the form $A \to s_1 \mid s_2 \mid \cdots \mid s_n$ to P' where $B \to s_1 \mid s_2 \mid \vec{\cdots} \mid s_n$. Note that each $s_i \in V \cup T$ and none of the s_i form unit productions.

Remove useless productions:
There are essentially two types of useless productions:

- Productions containing variables that do not terminate.
- Productions containing variables that are not reachable.

Identify these productions are remove them from P'.

Correct production rule format:
A context-free grammar G is in CNF, if the production rules have the following form:

- $A \to AB$ where A and B are variables, or
- $A \to a$ where a is a single terminal.

Example:
Take the following grammar:

$S \to ASA \mid aB$
$A \to B \mid S$
$B \to b \mid \varepsilon$

Step 1: Remove recursive start symbol

$S' \to S$
$S \to ASA \mid aB$
$A \to B \mid S$
$B \to b \mid \varepsilon$

Step 2: Remove ε-productions

$S' \to S$
$S \to ASA \mid aB \mid AS \mid SA \mid S \mid a$
$A \to B \mid S$
$B \to b$

Step 3: Remove unit productions

$S' \to ASA \mid aB \mid AS \mid SA \mid a$
$S \to ASA \mid aB \mid AS \mid SA \mid a$
$A \to b \mid ASA \mid aB \mid AS \mid SA \mid a$
$B \to b$

Step 4: Ensure correct production rule format

$S' \to AA1 \mid UB \mid AS \mid SA \mid a$
$S \to AA1 \mid UB \mid AS \mid SA \mid a$
$A \to b \mid AA1 \mid UB \mid AS \mid SA \mid a$
$A1 \to SA$
$U \to a$
$B \to b$

3.2.7 Proving languages are not context free

Pumping Lemma for CFLs

For every CFL A, there is a p such that if $s \in A$ and $|s| \geq p$ then $s = uvxyz$ where

1. $uv^ixy^iz \in A$ for all $i \geq 0$.
2. $vy \neq \varepsilon$.
3. $|vxy| \leq p$.

Informally, all long string in A are pumpable and stay in A.

Example 1:
Let $B = \{0^k1^k2^k | k \geq 0\}$. Show that B is not a CFL.

Assume, for the sake of contradiction, that B is a CFL.
The CFL pumping lemma gives p as above. Let $s = 0^p1^p2^p \in B$.
The pumping lemma says that we can divide $s = uvxyz$ satisfying the 3 conditions.
Condition 3 ($|vxy| \leq p$) implies that vxy cannot contain both 0s and 2s.
So $uv^2xy^2z \notin B$, violating condition 1, which is a contradiction.
Therefore, our assumption is false. We conclude that B is not a CFL.

Example 2:
Let $F = \{ww | w \in \Sigma^*\}$ where $\Sigma = \{0, 1\}$. Show that F is not a CFL.

Assum, for the sake of contradiction, that F is a CFL.
The CFL pumping lemma gives p as above. Take $s = 0^p1^p0^p1^p \in F$.
Condition 3 implies that vxy does not overlap two runs of 0s or two runs of 1s.
Therefore, in uv^2xy^2z, two runs of 0s or two runds of 1s have unequal length.
So $uv^2xy^2z \notin F$ violating Condition 1, which is a contradiction.
Thus, F is not a CFL.

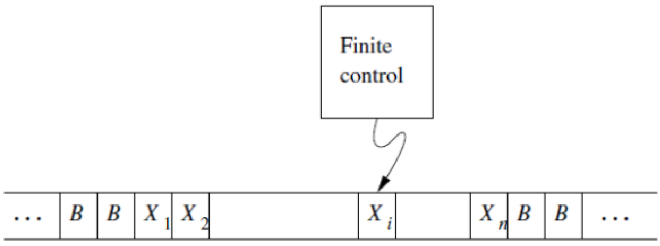
3.3 Turing Machines

3.3.1 Turing machines

Definition: Turing machines

A **turing machine** M is a 7-tuple $(Q, \Sigma, \Gamma, \delta, q_0, \square, F)$ where:

- Q is the set of states.
- Σ is the input alphabet *(finite)*.
- Γ is the tape alphabet *(finite)*.
- δ is the transition function.
- $q_0 \in Q$ is the initial/start state.
- $F \subseteq Q$ is the final state(s).
- $\square \in \Gamma$ is the blank symbol.



There some conditions:

- $\Sigma \subseteq \Gamma$, since the input is written on the tape initially.
- $\square \in \Gamma \setminus \Sigma$, since the rest of the tape is blank.

Initially, we're in state q_0. The tape contains input w surrounded by $\square$. The tape head is in the left-most cell of the input also denoted with D. The transitions work as follows:

- $\delta : Q \times \Gamma \Rightarrow Q \times \Gamma \times \{L, R\}$.
- $\delta(q, x) = (p, y, d)$ means that if M is in state q and the tape head is over symbol x, then M:
  1. Changes state to p.
  2. Replaces x by y on the tape.
  3. Moves the tape head by one cell in a direction d *(Either left or right)*.

3.3.2 Determinism

A turing machine is **deterministic** if, for each $\delta(q, x)$ we have at most one move *($\delta(q, x)$ could also be undefined)*.

Acceptance:
w is accepted by a turing machine M if M, when started with w on the tape, eventually enters a final state. We can assume that all final states are halting, i.e. no transition is defined for them.

Rejection:
M halts in non-final states. In other words, in states for which no transition is defined, or it never halts *(Infinite loop)*.

3.3.3 Instantaneous descriptions

This is the configuration of a turing machine. It describes the current situation of the turing machine (and the tape).

$$\text{ID} = \propto q\beta \text{ with } q \in Q, \propto_1, \propto_2 \in \Gamma^*$$

Meaning:

- The non-blank portion of the tape contains $\propto, \beta$.
- The head is on the left-most symbol of β.
- The machine is in state q.

We will use $\vdash$ and $\vdash^*$ to denote the changes.

3.3.4 Language accepted by a turing machine

Let $M = (Q, \Sigma, \Gamma, \delta, q_0, \square, F)$ be a turing machine. Then, the language accepted by M is

$$L(M) = \{w \in \Sigma^* | q_0 w \vdash^* \propto p\beta \text{ with } p \in F \text{ and } \propto_1 \beta \in \Gamma^*\}$$

The set of all languages we can accept using a turing machine is often called the recursively enumerable languages. Conversely, a turing machine can accept a language by halting:

$$H(M) = \{w \in \Sigma | q_0 q \vdash^* \propto p\beta \text{ where there is no possible move from } \propto p\beta\}$$